ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Báo cáo kỹ thuật

TransitPulse MBTA

Giám sát xe buýt thời gian thực và cảnh báo trễ dự kiến với Kafka, Spark Structured Streaming và Cassandra

---

Môn học:	Data Engineering	
Giảng viên:	PGS.TS VÕ THỊ NGỌC CHÂU	
Thành viên:	Nguyễn Quang Minh	2570784
	Trần Mậu Thật	2670373
	Trần Trung Anh	2670252
	Nguyễn Quỳnh Trang	2670382

Thành phố Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Giới thiệu **4**

1.1 Bối cảnh . . . 4

1.2 Mục tiêu . . . 4

1.3 Người dùng và câu hỏi nghiệp vụ . . . 5

1.4 Phạm vi . . . 5

1.5 Đóng góp chính . . . 5

1.6 Cấu trúc báo cáo . . . 6

2 Bài toán và yêu cầu **7**

2.1 Nguồn dữ liệu . . . 7

2.2 Thách thức kỹ thuật . . . 7

2.3 Yêu cầu chức năng . . . 8

2.4 Yêu cầu phi chức năng . . . 9

2.5 Định nghĩa và cách tính các chỉ số . . . 9

2.5.1 Độ mới của vị trí xe . . . 9

2.5.2 Chuyển giờ GTFS sang UTC . . . 10

2.5.3 Độ trễ dự kiến và phân loại uncertainty . . . 11

2.5.4 KPI độ trễ dự kiến theo tuyến . . . 11

2.5.5 Đối soát số lượng dữ liệu . . . 12

3 Kiến trúc và thiết kế hệ thống **13**

3.1 Tổng quan kiến trúc . . . 13

3.2 Công nghệ sử dụng . . . 14

3.3 Định danh và hợp đồng dữ liệu . . . 14

3.3.1 Định danh tất định . . . 14

3.3.2 Envelope Kafka . . . 15

3.4 Mô hình thời gian và đồng hồ . . . 15

3.5 Thu thập dữ liệu và hàng đợi gửi bền vững . . . 16

3.6 Vận chuyển qua Kafka . . . 17

3.7 Xử lý luồng với Spark Structured Streaming . . . 18
3.7.1 Nguồn Kafka và checkpoint . . . 18
3.7.2 Các bước trong một micro-batch . . . 18
3.7.3 Xác định lịch và phiên chuyến . . . 19
3.7.4 Tính KPI trên Spark . . . 19
3.8 Giao thức công bố generation . . . 20
3.8.1 Vấn đề . . . 20
3.8.2 Giải pháp: generation bất biến, manifest và pointer CAS . . . 20
3.9 Mô hình dữ liệu Cassandra . . . 23
3.9.1 Nguyên tắc thiết kế . . . 23
3.9.2 Các bảng chính . . . 23
3.9.3 Nhất quán khi đọc . . . 25
3.10 Lưu trữ lịch sử và báo cáo batch . . . 25
3.11 Tầng phục vụ FastAPI . . . 25
3.12 Giao diện người dùng . . . 26
3.13 Replay và dữ liệu synthetic . . . 27
3.14 Vận hành, triển khai và quan sát . . . 27
3.14.1 Môi trường chạy trực tiếp . . . 27
3.14.2 Truy cập và an toàn . . . 28
3.14.3 Quan sát . . . 28

4 Kiểm chứng và đánh giá thực nghiệm 29

4.1 Chiến lược kiểm chứng . . . 29
4.1.1 Các tầng kiểm thử . . . 29
4.1.2 Golden fixture và oracle độc lập . . . 29
4.1.3 Kịch bản golden . . . 30
4.1.4 Kịch bản tích hợp . . . 30
4.2 Môi trường thực nghiệm . . . 31
4.3 Kết quả . . . 32
4.3.1 Bootstrap và hạ tầng . . . 32

4.3.2 Kiểm thử unit, runtime và Spark . . . 32
4.3.3 Golden smoke end-to-end . . . 33
4.3.4 Thu thập trực tiếp dữ liệu MBTA . . . 33
4.3.5 Truy vết dữ liệu và độ trễ công bố . . . 34
4.3.6 Giao diện trong phiên thu thập trực tiếp . . . 35
4.4 Benchmark . . . 38
4.4.1 Phương pháp . . . 38
4.4.2 Kết quả stress . . . 39
4.4.3 Kết quả phục hồi . . . 41
4.5 Tổng hợp mức độ đáp ứng . . . 42

5 Kết luận **43**
5.1 Tổng kết . . . 43
5.2 Hạn chế . . . 43
5.3 Hướng phát triển . . . 44

6 Tài liệu tham khảo **46**

1 Giới thiệu

1.1 Bối cảnh

Để theo dõi hoạt động của xe buýt, hệ thống cần kết hợp lịch trình với dữ liệu vận hành thực tế. Lịch trình tĩnh theo chuẩn GTFS Schedule [2] cung cấp thông tin về tuyến, trạm, chuyến, giờ chạy và hình dạng tuyến đường. Dữ liệu theo chuẩn GTFS Realtime [3] bổ sung ba luồng cập nhật, gọi là *feed*: vị trí xe (Vehicle Positions), dự báo giờ đến và rời trạm (Trip Updates), cùng cảnh báo dịch vụ (Service Alerts). Massachusetts Bay Transportation Authority (MBTA), đơn vị vận hành giao thông công cộng tại khu vực Boston, công bố hai loại dữ liệu này dưới dạng tệp ZIP và JSON để người dùng có thể truy cập công khai [1].

Dữ liệu thời gian thực được cập nhật liên tục nhưng có thể thiếu trường, đến trễ, ngừng cập nhật hoặc lặp lại nội dung đã nhận. Vì vậy, chỉ hiển thị phản hồi mới nhất từ nguồn là chưa đủ. Hệ thống còn cần làm rõ:

- Vị trí xe được ghi nhận lúc nào và còn đủ mới để sử dụng hay không?
- Giờ đến trạm là dự báo từ dữ liệu thời gian thực hay chỉ là giờ theo lịch trình?
- Kết quả đang hiển thị đã được xử lý đầy đủ hay còn thiếu do quá trình ghi dữ liệu bị gián đoạn?

Từ các yêu cầu trên, nhóm xây dựng TransitPulse MBTA trong khuôn khổ đồ án môn Data Engineering. Hệ thống sử dụng Apache Cassandra [8] để lưu trữ và quản lý dữ liệu, Apache Spark Structured Streaming [6, 5] để xử lý dữ liệu luồng, Apache Kafka để chuyển các sự kiện giữa các thành phần và FastAPI để cung cấp kết quả cho giao diện người dùng.

1.2 Mục tiêu

Mục tiêu của dự án là xây dựng một quy trình hoàn chỉnh: thu thập dữ liệu xe buýt MBTA, xử lý qua chuỗi **Kafka → Spark Structured Streaming → Cassandra**, sau đó cung cấp kết quả qua FastAPI cho giao diện React, bảng theo dõi Streamlit và notebook trên Google Colab. Bên cạnh việc hiển thị kết quả, hệ thống cần đáp ứng các yêu cầu về độ tin cậy của dữ liệu:

- **Quản lý phiên bản**: mỗi kết quả được gắn với một lần chạy (*run*), một bản chụp dữ liệu nguồn (*snapshot*) và một phiên bản dữ liệu phục vụ truy vấn (*generation*). Khi đã tạo, nội dung của generation không thay đổi.
- **Phân biệt các mốc thời gian**: thời điểm ghi nhận vị trí xe, thời điểm tạo feed và thời điểm hệ thống xử lý được lưu riêng để tránh nhầm lẫn khi đánh giá độ mới của dữ liệu.
- **Kiểm soát chất lượng**: các bản ghi bị loại, cách ly hoặc xác định là trùng lặp đều có mã lý do và tham chiếu đến dữ liệu gốc.
- **Phục hồi khi có lỗi**: quá trình xử lý lại cần giữ nguyên kết quả logic, tránh hiển thị dữ liệu ghi dở, đếm trùng KPI hoặc thay thế trạng thái mới bằng dữ liệu cũ.

- **Truy vết nguồn gốc**: từ kết quả trên API có thể xác định Kafka offset, tệp dữ liệu gốc và giá trị kiểm tra toàn vẹn (checksum) của lần thu thập tương ứng.

1.3 Người dùng và câu hỏi nghiệp vụ

Bảng 1 liệt kê bốn nhóm người dùng và thông tin mà mỗi nhóm cần.

Bảng 1: Người dùng mục tiêu của TransitPulse

Vai trò	Câu hỏi cần trả lời	Thông tin hệ thống cung cấp
Hành khách	Xe ở đâu, khi nào tới trạm, tuyến có gián đoạn không	Bản đồ xe theo tuyến, ETA theo trạm có nhãn chất lượng, cảnh báo dịch vụ
Người theo dõi vận hành	Tuyến nào có dự báo chậm, bao nhiêu xe có vị trí mới	KPI "độ trễ dự kiến" kèm tử số, mẫu số và thời điểm tính
Kỹ sư dữ liệu	Nguồn còn cập nhật không, lỗi ở bước nào, có dữ liệu chờ xử lý không	Trang chất lượng và trạng thái xử lý, hàng đợi lỗi (DLQ), tham chiếu dữ liệu gốc và thông tin truy vết
Người đánh giá	Dữ liệu có đi qua Kafka, Spark, Cassandra và cho kết quả đúng không	Kiểm thử golden smoke, báo cáo kiểm thử, kịch bản lỗi và kết quả đo hiệu năng

1.4 Phạm vi

Phiên chạy thử đã thực hiện trên **10 tuyến xe buýt** của MBTA (`route_type=3`): `1`, `15`, `22`, `23`, `28`, `32`, `39`, `57`, `66` và `77`. Danh sách tuyến được đối chiếu với GTFS tại thời điểm chạy. Mỗi lần chạy sử dụng một phiên bản GTFS cố định. Hệ thống hỗ trợ ba chế độ, cùng đi qua một quy trình xử lý:

- **live**: thu thập trực tiếp ba feed MBTA theo chu kỳ 30 giây;
- **replay**: phát lại dữ liệu đã ghi ở tốc độ 1x, 10x hoặc 50x, có thể tạm dừng và tiếp tục;
- **synthetic**: phát dữ liệu mô phỏng được sinh theo quy tắc tất định. Dữ liệu này mang nhãn `source_type=synthetic` và không được hiển thị như dữ liệu xe MBTA thực tế.

Dự án chưa xây dựng mô hình dự báo giờ đến trạm (ETA) riêng, chưa đánh giá độ chính xác ETA do thiếu dữ liệu thực tế làm đối chứng (ground truth), và chưa thực hiện tối ưu hành trình hay đếm hành khách. Hạ tầng có tính sẵn sàng cao và triển khai trên nhiều vùng cũng nằm ngoài phạm vi đồ án. ETA hiển thị được lấy từ dự báo của MBTA. Trong báo cáo, "độ trễ dự kiến" được hiểu là chênh lệch giữa giờ dự báo và giờ theo lịch GTFS; chỉ số này không thể hiện độ trễ thực tế hoặc chỉ số đúng giờ chính thức của MBTA.

1.5 Đóng góp chính

1. Xây dựng quy trình xử lý từ đầu đến cuối trên máy ảo Google Colab. Các thành phần được chạy trực tiếp bằng tệp nhị phân, không cần Docker hay quyền sudo, gồm Kafka 4.2.2 (KRaft), Cassandra 5.0.9, Spark 3.5.7, bộ thu thập dữ liệu, FastAPI và ba loại giao diện.

2. Thiết kế cơ chế công bố dữ liệu dựa trên generation bất biến. Bản mô tả generation (manifest) ghi nhận trạng thái `BUILDING`/`READY`, còn con trỏ `feed_current` được cập nhật bằng compare-and-set, sử dụng lightweight transaction của Cassandra. Trong điều kiện không có giao dịch chung giữa Kafka, hệ thống tệp và Cassandra, cách làm này hỗ trợ giao nhận ít nhất một lần (at-least-once), đồng thời cho phép ghi và công bố lại mà không thay đổi kết quả logic (idempotent).

3. Xây dựng các mã định danh tất định (`snapshot_id`, `observation_id`, `prediction_id`, ...), cho kết quả thống nhất trên Python và Spark. Nhờ đó, việc thử lại hoặc phát lại dữ liệu không tạo thêm bản ghi trùng.

4. Phân biệt đồng hồ nguồn, đồng hồ hệ thống và đồng hồ nghiệp vụ; xử lý giờ GTFS vượt 24:00 và ngày chuyển đổi giờ mùa hè (DST).

5. Tính KPI độ trễ dự kiến theo quy tắc chọn một trạm mục tiêu (*target stop*) cho mỗi chuyến. Kết quả đi kèm tử số, mẫu số, các lý do loại trừ và thời điểm tính để người đọc có thể kiểm tra.

6. Xây dựng nhiều lớp kiểm chứng: bộ dữ liệu mẫu có kết quả kỳ vọng được xác định thủ công (golden fixture), bộ tính kết quả đối chứng độc lập (oracle) chỉ dùng thư viện chuẩn Python, các kịch bản chủ động gây lỗi (fault injection) và bộ công cụ đo hiệu năng (benchmark) chỉ ghi nhận số đo thực tế.

1.6 Cấu trúc báo cáo

Phần 2 trình bày nguồn dữ liệu, các khó khăn kỹ thuật, yêu cầu hệ thống và cách tính các chỉ số. Phần 3 mô tả kiến trúc cùng thiết kế của từng thành phần, từ thu thập, chuyển và xử lý dữ liệu đến lưu trữ trên Cassandra, công bố kết quả và vận hành. Phần 4 trình bày phương pháp kiểm chứng, kết quả chạy thực tế, giao diện và các phép đo hiệu năng trên Google Colab. Cuối cùng, Phần 5 tổng kết kết quả, nêu hạn chế và đề xuất hướng phát triển. Mã nguồn của dự án được công bố tại [13].

2 Bài toán và yêu cầu

2.1 Nguồn dữ liệu

Dự án sử dụng bốn nguồn dữ liệu công khai của MBTA, được liệt kê trong Bảng 2. Các nguồn này cung cấp cả lịch trình và thông tin vận hành theo thời gian thực. Hệ thống không phụ thuộc vào MBTA V3 API hay nguồn dữ liệu khác; khi xảy ra lỗi mạng, dữ liệu MBTA cũng không được thay bằng dữ liệu từ nguồn khác.

Bảng 2: Bốn nguồn dữ liệu MBTA được sử dụng

Nguồn	URL	Vai trò
GTFS Static	https://cdn.mbta.com/MBTA_GTFS.zip	Tuyến, trạm, chuyến, lịch trình, hình học tuyến và múi giờ
Vehicle Positions	https://cdn.mbta.com/realtime/VehiclePositions.json	Vị trí xe, thông tin nhận diện chuyến và thời điểm quan sát
Trip Updates	https://cdn.mbta.com/realtime/TripUpdates.json	Giờ dự báo đến/rời trạm và trường `uncertainty`
Service Alerts	https://cdn.mbta.com/realtime/Alerts.json	Cảnh báo dịch vụ, đối tượng áp dụng và thời hạn hiệu lực

GTFS Static lưu thông tin lịch trình trong các bảng `routes`, `trips`, `stop_times`, `stops`, `calendar`, `calendar_dates` và `shapes`. Trong khi đó, mỗi luồng dữ liệu GTFS Realtime (feed) gồm phần `header` và danh sách `entity`. Header chứa phiên bản, `incrementality` và `timestamp` của feed; nội dung entity phụ thuộc vào loại feed.

Cụ thể, Vehicle Positions cung cấp mã xe, tọa độ, `timestamp` của lần đo và thông tin nhận diện chuyến. Trip Updates chứa danh sách `stop_time_update`, trong đó mỗi phần tử có giờ dự báo đến trạm (arrival), rời trạm (departure), hoặc cả hai, cùng trường `uncertainty`. Service Alerts dùng `informed_entity` để xác định đối tượng áp dụng theo tuyến, trạm, chuyến, đơn vị vận hành hoặc loại tuyến; `active_period` xác định thời gian cảnh báo có hiệu lực.

Trong dữ liệu quan sát ngày 06/10/2026, feed Trip Updates của MBTA **không có trường** `delay`. Vì vậy, hệ thống tính độ trễ dự kiến bằng cách lấy giờ dự báo trừ giờ lịch GTFS. Hai mốc được so sánh phải thuộc **cùng loại sự kiện**: giờ đến so với giờ đến, giờ rời so với giờ rời.

2.2 Thách thức kỹ thuật

Dữ liệu nguồn có thể thiếu hoặc không hợp lệ. Một entity có thể thiếu `vehicle.id`, có tọa độ ngoài miền hợp lệ hoặc có `timestamp` ở tương lai. Thời gian cũng có thể được ghi theo mili giây thay vì giây. Ngoài ra, một chuyến có thể đi qua cùng một trạm hai lần, nên chỉ dùng `stop_id` là chưa đủ để ghép dự báo với lịch trình.

Dữ liệu có thể được xử lý nhiều lần. Với cơ chế giao nhận at-least-once, Kafka và checkpoint của Spark hỗ trợ tránh mất thông điệp, nhưng một thông điệp vẫn có thể được xử lý nhiều lần. Chẳng hạn, bộ thu thập

(collector) có thể gửi lại dữ liệu sau khi khởi động lại, hoặc Spark có thể xử lý lại một lô nhỏ (micro-batch) sau sự cố. Một bản chụp dữ liệu tại thời điểm thu thập (snapshot) cũ cũng có thể đến sau snapshot mới. Trong các trường hợp này, kết quả logic của hệ thống cần được giữ nhất quán.

Không có giao dịch chung giữa các thành phần. Trong một lần xử lý, hệ thống ghi dữ liệu vào Kafka qua DLQ, vào hệ thống tệp dưới dạng curated Parquet và vào nhiều bảng Cassandra. Ba nơi lưu trữ này không có cơ chế xác nhận ghi nguyên tử chung. Vì vậy, hệ thống cần kiểm soát thời điểm đưa một phiên bản dữ liệu (generation) vào phục vụ, tránh để API đọc dữ liệu khi quá trình ghi chưa hoàn tất.

Phân biệt các mốc thời gian. Thời điểm đo vị trí xe, tạo feed, thu thập, gửi lên Kafka và hiển thị dữ liệu là các mốc khác nhau. Lịch GTFS còn cho phép giờ vượt 24:00 để biểu diễn các chuyến qua nửa đêm. Vào ngày đổi giờ mùa hè, cách cộng trực tiếp giờ lịch vào mốc nửa đêm có thể cho kết quả lệch một giờ. Khi phát lại dữ liệu (replay), thời điểm hiện tại được xác định theo đồng hồ của bản ghi lịch sử, thay vì giờ máy.

Thiết kế dữ liệu theo truy vấn của Cassandra. Cassandra không hỗ trợ phép nối bảng. Truy vấn ngoài khóa phân vùng có thể cần `ALLOW FILTERING` và phải quét toàn bảng. Do đó, mô hình dữ liệu được xây dựng từ nhu cầu truy vấn (query-first), để mỗi endpoint đọc theo phân vùng và giới hạn số bản ghi bằng `LIMIT`.

2.3 Yêu cầu chức năng

Bảng 3 trình bày 12 yêu cầu chức năng cùng các thành phần đảm nhận từng yêu cầu.

Bảng 3: Yêu cầu chức năng và thành phần thực hiện

Mã	Yêu cầu	Thành phần
FR01	Nhập tệp GTFS ZIP; dùng SHA-256 của tệp làm phiên bản; lưu bảng chiều Parquet và metadata bất biến	`static/`
FR02	Thu thập độc lập 3 feed; lưu dữ liệu gốc và metadata trước khi gửi; không tạo snapshot rỗng khi hết thời gian chờ	`ingestion/collector.py`, `archive.py`
FR03	Gửi snapshot qua Kafka với schema, checksum và ID ổn định; lưu bền vững hàng đợi gửi (outbox)	`ingestion/outbox.py`, `kafka_producer.py`
FR04	Dùng Spark để phân tích, kiểm tra, tách phần tử, nối với GTFS và loại trùng; gắn mã chất lượng và tham chiếu dữ liệu gốc	`processing/`
FR05	Lấy vị trí xe từ generation hiện tại; phân lớp fresh/stale/unknown/invalid; không cộng dồn số xe qua nhiều snapshot	`processing/vehicles.py`, `api/freshness.py`
FR06	Cung cấp dự báo đến/rời trạm, uncertainty của MBTA và độ trễ dự kiến khi xác định được lịch	`processing/predictions.py`, `schedule.py`
FR07	Tính KPI tuyến với tối đa một trạm mục tiêu cho mỗi chuyến trong ngày dịch vụ; công bố tử số, mẫu số, lý do loại trừ và thời điểm tính	`processing/kpi.py`
FR08	Xác định đối tượng và thời hạn của cảnh báo; phân biệt cảnh báo MBTA với cảnh báo chất lượng pipeline	`processing/alerts.py`, API `/alerts`
FR09	Phân trang lịch sử xe; lưu dữ liệu đã xử lý dưới dạng curated Parquet; lập báo cáo lịch sử bằng Spark batch	`processing/report_job.py`
FR10	Giữ generation bất biến; API chỉ đọc READY; dữ liệu trùng, đến muộn hoặc xử lý lại không làm tăng KPI hay lùi con trỏ	`processing/batch.py`, `storage/pointer.py`

Mã	Yêu cầu	Thành phần
FR11	Phát lại với đúng phiên bản dữ liệu tĩnh ở tốc độ 1x/10x/50x, có tạm dừng/tiếp tục; tách đồng hồ nguồn khỏi đồng hồ máy	`replay/`
FR12	Cung cấp API, dashboard, trang chất lượng; kiểm thử golden, kiểm thử lỗi và benchmark có bằng chứng	`api/`, `app/`, `tests/`, `benchmarks/`

2.4 Yêu cầu phi chức năng

Bảng 4: Yêu cầu phi chức năng

Mã	Thuộc tính	Cách đáp ứng
NFR01	Tính đúng	Bộ dữ liệu chuẩn (golden fixture) viết tay và bộ tính kết quả đối chiếu (oracle) độc lập `tests/oracle/`
NFR02	Phục hồi	Checkpoint của Spark, ghi đầu ra lặp lại không làm đổi kết quả (idempotent), manifest cố định và outbox bền vững
NFR03	Độ trễ	Mục tiêu p95 của độ trễ đưa dữ liệu vào phục vụ (visibility latency) < 10 s; cần kiểm chứng bằng benchmark
NFR04	Dung lượng	Gói thông điệp (envelope) ≤ 4 MiB, thời gian lưu topic 6 giờ, dung lượng dành cho dữ liệu gốc 2 GiB
NFR05	Truy vết	Lần theo nguồn gốc dữ liệu từ API về manifest, Kafka offset và checksum của dữ liệu gốc
NFR06	Tái lập	Cố định phiên bản bằng `==`, dùng seed tất định và manifest môi trường chạy có checksum
NFR07	Theo dõi vận hành	Log JSON có cấu trúc, chỉ số Prometheus và `/pipeline/status`
NFR08	Giới hạn truy vấn	Không dùng `ALLOW FILTERING`; mọi truy vấn đọc có `LIMIT` và phân trang bằng token
NFR09	An toàn demo	Các cổng chỉ lắng nghe tại `127.0.0.1`; giới hạn nguồn truy cập CORS; không công khai Kafka/Cassandra
NFR10	Demo ngoại tuyến	Phát lại bản ghi và chạy dữ liệu tổng hợp (synthetic) khi không có kết nối tới MBTA

2.5 Định nghĩa và cách tính các chỉ số

Phần này trình bày cách tính và điều kiện sử dụng các chỉ số của hệ thống. Các ngưỡng được xác định theo **chính sách của dự án** (policy) và được lưu cố định trong `run_metadata.policy_json` khi tạo một lần chạy (run). Đây là các quy tắc phục vụ dự án, không phải cam kết mức dịch vụ (SLA) của MBTA.

2.5.1 Độ mới của vị trí xe

Gọi c là thời gian theo đồng hồ nghiệp vụ (application clock) tại lúc đánh giá và t_{obs} là thời điểm nguồn đo vị trí xe. Tuổi của quan sát, tức khoảng thời gian đã trôi qua kể từ lần đo, được tính bằng

$$a = c - t_{obs}. \tag{1}$$

Hệ thống dùng ngưỡng dữ liệu cũ (stale) $\theta_v = 120$ giây và cho phép lệch về tương lai tối đa $\epsilon = 5$ giây. Trạng thái vị trí của mỗi xe được xác định như sau:

$$\text{class}(a) = \begin{cases} \text{invalid} & \text{nếu vị trí không hợp lệ, hoặc } a < -\epsilon \text{ (FUTURE_TIMESTAMP)}, \\ \text{unknown} & \text{nếu } t_{obs} \text{ không có } (a = \text{null}), \\ \text{fresh} & \text{nếu } -\epsilon \le a \le \theta_v \ (a < 0 \text{ được gắn cờ CLOCK_SKEW và hiển thị 0}), \\ \text{stale} & \text{nếu } a > \theta_v. \end{cases} \tag{2}$$

Timestamp lớn hơn 10^{11} được xem là sai đơn vị, gắn mã `INVALID_TIMESTAMP_UNIT` và không được tự động quy đổi. Trạng thái độ mới (freshness) được API tính lại trong mỗi yêu cầu thay vì lưu cố định trong Cassandra. Nhờ đó, khi feed ngừng cập nhật, vị trí xe vẫn chuyển sang stale dù hệ thống không nhận thêm thông điệp. Hàm phân lớp trong `contracts/policy.py` được trình bày dưới đây:

```
def vehicle_freshness(*, position_valid, observation_time, application_clock, policy):
    if not position_valid:
        return FreshnessResult(FreshnessClass.INVALID, None, ())
    if observation_time is None:
        return FreshnessResult(FreshnessClass.UNKNOWN, None, ())
    age = (application_clock - observation_time).total_seconds()
    if age < -policy.future_tolerance_seconds:
        return FreshnessResult(FreshnessClass.INVALID, None, ("FUTURE_TIMESTAMP",))
    if age < 0:
        return FreshnessResult(FreshnessClass.FRESH, 0.0, ("CLOCK_SKEW",))
    if age <= policy.vehicle_stale_seconds:
        return FreshnessResult(FreshnessClass.FRESH, age, ())
    return FreshnessResult(FreshnessClass.STALE, age, ())
```

Đối với từng feed, ngưỡng stale được tính theo thời gian đã trôi qua kể từ timestamp trong header: 120 giây cho Vehicle Positions, 90 giây cho Trip Updates và 300 giây cho Alerts. Tài liệu GTFS Realtime Best Practices khuyến nghị tuổi dữ liệu tối đa là 90 giây cho VP/TU và 10 phút cho Alerts [4].

2.5.2 Chuyển giờ GTFS sang UTC

GTFS biểu diễn giờ bằng định dạng `HH:MM:SS`, tính từ mốc của ngày dịch vụ; giá trị `HH` có thể lớn hơn hoặc bằng 24. Theo chuẩn GTFS, mốc này được xác định bằng cách lấy "trưa giờ địa phương trừ 12 giờ" [2]. Cách xác định đó áp dụng cả trong ngày đổi giờ mùa hè:

$$s_{gtfs} = 3600 \cdot HH + 60 \cdot MM + SS, \tag{3}$$

$$t_{anchor} = \text{UTC}\big(\text{trưa địa phương của ngày dịch vụ } D\big) - 12\,\text{h}, \tag{4}$$

$$t_{sched} = t_{anchor} + s_{gtfs}. \tag{5}$$

Ví dụ, trong múi giờ `America/New_York`, ngày 2026-03-08 là ngày chuyển sang EDT. Giờ GTFS `01:30:00` tương ứng với `2026-03-08T05:30:00Z`. Nếu cộng trực tiếp số giờ vào mốc nửa đêm mà không xét quy tắc trên, kết quả sẽ lệch một giờ. Trên Spark, hệ thống tính `to_utc_timestamp('<date> 12:00:00', tz)` rồi trừ 43 200 giây, đồng thời bắt buộc đặt `spark.sql.session.timeZone=UTC`.

2.5.3 Độ trễ dự kiến và phân loại uncertainty

Với một sự kiện e là đến trạm (arrival) hoặc rời trạm (departure), độ trễ dự kiến chỉ được tính khi đã xác định được lịch của lần ghé trạm tương ứng:

$$d_e = t_e^{pred} - t_e^{sched}, \tag{6}$$

trong công thức này, t_e^{sched} là giờ lịch của **cùng loại sự kiện** với giờ dự báo. Giá trị d_e âm cho biết xe được dự báo đến hoặc rời sớm hơn lịch. Nếu ngày dịch vụ còn mơ hồ (ambiguous) hoặc chưa xác định được (unresolved), hệ thống đặt $d_e = \text{null}$ và loại sự kiện khỏi KPI. Khi đó, giờ dự báo do nguồn cung cấp vẫn được hiển thị.

Trường `uncertainty` trong dữ liệu xe buýt MBTA có cách diễn giải riêng [1]. Bộ chuyển đổi `mbta_bus_uncertainty_v1` phân loại giá trị u theo các trường hợp sau:

$$\text{uc}(u) = \begin{cases} \text{unknown_uncertainty} & u \text{ không có (vẫn eligible, đếm riêng)}, \\ \text{realtime} & 0 \le u < 300, \\ \text{schedule_only} & u = 300, \\ \text{degraded} & u = 301, \\ \text{invalid} & u < 0 \text{ hoặc } u > 301. \end{cases} \tag{7}$$

2.5.4 KPI độ trễ dự kiến theo tuyến

KPI được tính theo *trip instance*, tức một `trip_id` trong một ngày dịch vụ, thay vì tính trên từng dự báo. Với mỗi trip instance, chính sách `first_future_resolved_stop_v1` chọn tối đa một sự kiện mục tiêu (*target event*). Cụ thể, hệ thống xét các sự kiện có thời gian sau `metrics_as_of`, đã xác định được trạm và không mang quan hệ `SKIPPED/NO_DATA`, rồi chọn sự kiện có `stop_sequence` nhỏ nhất. Tại mỗi lần ghé trạm, giờ đến được ưu tiên; nếu thiếu thì dùng giờ rời.

Sau khi chọn sự kiện mục tiêu, hệ thống chỉ đánh giá tính hợp lệ của sự kiện đó. Nếu chất lượng dự báo không đạt yêu cầu, hệ thống không chuyển sang trạm tiếp theo để thay thế.

Nếu một trip instance không đủ điều kiện tính KPI, hệ thống ghi nhận **lý do đầu tiên khớp** trong thứ tự cố định sau: `CANCELED_TRIP`, `ADDED_TRIP`, `AMBIGUOUS_SERVICE_DATE`, `UNRESOLVED_TRIP`, `UNRESOLVED_STOP_SEQUENCE`, `NO_TARGET_STOP`, `SCHEDULE_ONLY_PREDICTION`, `DEGRADED_PREDICTION`, `INVALID_PREDICTION`, `NO_SCHEDULED_EVENT`. Gọi E_r là tập trip instance đủ điều kiện (eligible) của tuyến r và d_i là độ trễ tại sự kiện mục tiêu của chuyến i. Với ngưỡng trễ $\theta_{late} = 300$ giây, các chỉ số được tính như sau:

$$L_r = \{\, i \in E_r : d_i \ge \theta_{late} \,\}, \tag{8}$$

$$\text{predicted_late_rate}_r = \begin{cases} |L_r| \,/\, |E_r| & |E_r| > 0, \\ \text{null} & |E_r| = 0, \end{cases} \tag{9}$$

$$\text{mean_predicted_delay}_r = \frac{1}{|E_r|} \sum_{i \in E_r} d_i \quad (\text{null nếu } |E_r| = 0). \tag{10}$$

Khi không có chuyến đủ điều kiện, mẫu số bằng 0 và tỷ lệ nhận giá trị `null`. Giao diện hiển thị "Chưa có mẫu

đủ điều kiện" thay cho 0%, để tránh hiểu nhầm rằng tất cả các chuyến đều không trễ. Mỗi KPI được công bố kèm thời điểm `metrics_as_of`, tử số, mẫu số, `excluded_trip_counts` và `policy_version`.

Ví dụ với golden fixture trên tuyến synthetic R_A. Tại thời điểm 12:00:30Z, hệ thống xét hai chuyến. Với T1, sự kiện mục tiêu là đến S3 (`stop_sequence` 3), có giờ dự báo 12:11:00Z và giờ lịch 12:05:00Z, nên $d = 360$ s. Với T2, sự kiện tại `stop_sequence` 1 lúc 12:00:00Z đã qua. Vì vậy, hệ thống chọn sự kiện đến S3 (`stop_sequence` 2), có giờ dự báo 12:06:00Z và giờ lịch 12:05:00Z, nên $d = 60$ s.

Cả hai chuyến đều đủ điều kiện tính KPI. Khi đó, $|E| = 2$, $|L| = 1$, tỷ lệ trễ dự kiến bằng 0,5 và độ trễ dự kiến trung bình bằng $(360 + 60)/2 = 210$ s. Dù có nhiều dự báo đến và rời tại các trạm, mỗi chuyến vẫn chỉ đóng góp tối đa 1 vào mẫu số.

2.5.5 Đối soát số lượng dữ liệu

Trước khi đánh dấu một generation là READY, hệ thống đối soát số lượng dữ liệu qua các đẳng thức dưới đây. Hai vế của mỗi đẳng thức phải dùng cùng đơn vị đếm:

$$\begin{aligned}
\text{selected_entities} &= \text{fully_valid} + \text{partially_valid} + \text{rejected},\\
\text{valid_candidate_observations} &= \text{unique_observations} + \text{duplicate_observations},\\
\text{membership}_{(route)} &= \text{fresh} + \text{stale} + \text{unknown_age} + \text{invalid_có_ID},\\
\text{candidate_predictions} &= \text{retained} + \text{rejected} + \text{not_applicable},\\
\text{trip_instances} &= \text{eligible} + \sum \text{excluded}.
\end{aligned}$$

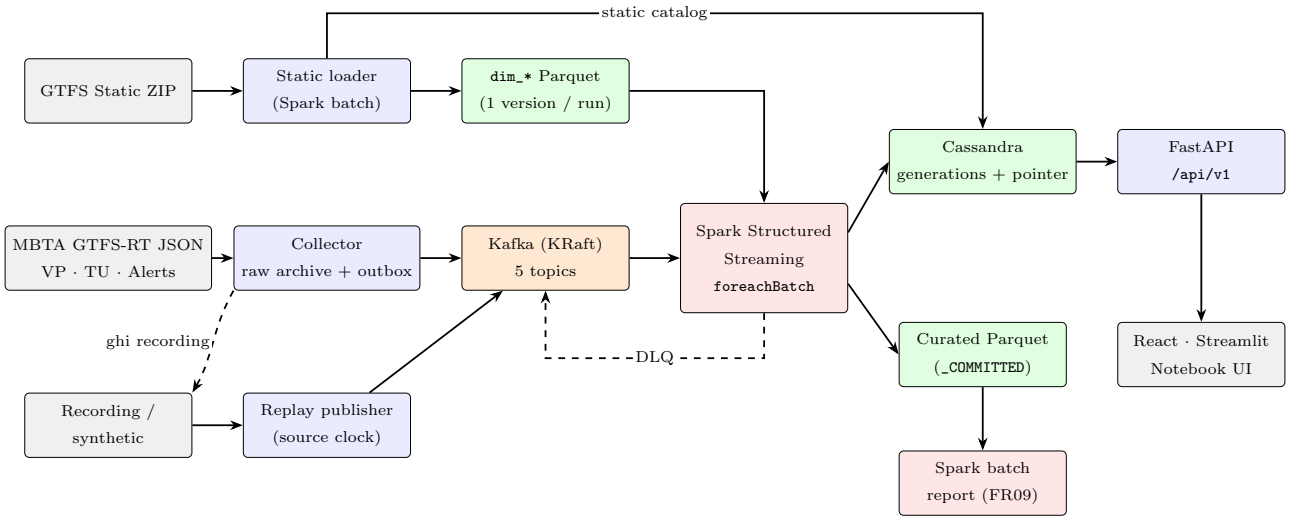
Một bản ghi có thể mang nhiều cảnh báo, nên tổng số cảnh báo không được dùng làm mẫu số khi tính tỷ lệ lỗi. Danh mục `QUALITY_REASONS` trong `contracts/enums.py` gồm 43 mã lý do. Mỗi mã xác định mức độ nghiêm trọng, cách xử lý mặc định, đơn vị đếm và nhóm tương ứng.

3 Kiến trúc và thiết kế hệ thống

3.1 Tổng quan kiến trúc

Kiến trúc tổng thể của hệ thống được trình bày trong Hình 1, gồm hai luồng dữ liệu chính. **Luồng dữ liệu tĩnh** xử lý tệp GTFS ZIP bằng Spark batch để tạo các bảng dữ liệu tham chiếu (dimension) ở dạng Parquet và danh mục dữ liệu tĩnh trong Cassandra. **Luồng dữ liệu thời gian thực** thu thập ba luồng cập nhật (feed) JSON, chuyển qua Kafka rồi xử lý bằng Spark Structured Streaming. Kết quả được lưu thành các phiên bản dữ liệu phục vụ truy vấn bất biến (generation) trong Cassandra và dữ liệu lịch sử đã xử lý (curated) ở dạng Parquet.

Dữ liệu phát lại (replay) và dữ liệu mô phỏng (synthetic) được gửi vào cùng topic Kafka với dữ liệu trực tiếp (live), nên cả ba chế độ dùng chung một quy trình xử lý. Giao diện lấy kết quả thông qua FastAPI; việc đọc Kafka, dữ liệu gốc (raw) và các feed MBTA được thực hiện ở các tầng phía sau.



Hình 1: *Kiến trúc tổng thể của TransitPulse MBTA. Mũi tên nét đứt là luồng phụ (DLQ, ghi recording).*

Bảng 5 tóm tắt luồng đi và mục đích sử dụng của từng nhóm dữ liệu.

Bảng 5: Các đường dữ liệu trong hệ thống

Nhóm	Luồng xử lý	Mục đích
Dữ liệu tĩnh	ZIP → phiên bản SHA-256 → Spark batch kiểm tra → `dim_*` Parquet + bảng tĩnh Cassandra	Lịch, tuyến, trạm, hình học tuyến theo phiên bản bất biến
Thời gian thực	JSON → raw + outbox → Kafka → Spark → Cassandra + curated	Vị trí xe, dự báo và cảnh báo
Phục vụ	Cassandra → FastAPI → React / Streamlit / notebook	Cung cấp dữ liệu đã xử lý
Phát lại	Recording + đúng phiên bản GTFS → publisher → cùng Kafka/Spark/API	Demo lặp lại, chạy ngoại tuyến và benchmark
Chất lượng	Lỗi phân tích/kiểm tra → DLQ (trích đoạn ≤ 2 KiB + tham chiếu dữ liệu gốc) + `quality_by_snapshot`	Tra cứu và điều tra lỗi
Báo cáo batch	Curated Parquet → Spark batch (loại trùng, cửa sổ thời gian) → báo cáo	Phân tích dữ liệu lịch sử

3.2 Công nghệ sử dụng

Các thư viện phụ thuộc, image và gói cài đặt đều được cố định phiên bản để việc triển khai có thể lặp lại. Bảng 6 liệt kê các công nghệ được sử dụng; hệ thống không dùng nhãn `latest`.

Bảng 6: Bộ công nghệ và phiên bản

Thành phần	Công nghệ (phiên bản)	Vai trò
Ngôn ngữ lõi	Python 3.11.17 (quản lý bởi uv 0.12.23)	Toàn bộ backend
JVM	Eclipse Temurin JDK 17.0.20.1+1	Kafka, Cassandra, Spark
Xử lý dữ liệu	Spark/PySpark 3.5.7 (Scala 2.12), `spark-sql-kafka-0-10_2.12:3.5.7`	Xử lý dữ liệu tĩnh, luồng và báo cáo batch
Trung chuyển	Apache Kafka 4.2.2 (KRaft, 1 node)	Vận chuyển snapshot, control, DLQ
CSDL phục vụ	Apache Cassandra 5.0.9 (1 node, RF1)	Generation, pointer, lịch sử, danh mục tĩnh
Client	confluent-kafka 2.15.1, cassandra-driver 3.30.1, httpx 0.28.1	Gửi Kafka, truy vấn CQL, thu thập HTTP
API	FastAPI 0.142.2, Pydantic 2.13.5, uvicorn 0.54.0	REST `/api/v1`
Giao diện	React 18 + MapLibre, Streamlit 1.65.0 + PyDeck 0.9.3, ipywidgets	Console, dashboard, notebook UI
Kiểm thử	pytest 9.1.1, Ruff 0.16.10, nbformat 5.11.1	Kiểm thử, kiểm tra mã và notebook

Hệ thống sử dụng Python 3.11 do `cassandra-driver` cần module `asyncore` khi không có `libev`, trong khi Python 3.12 đã loại bỏ module này. Phiên bản này cũng phù hợp với Spark 3.5.7, vốn hỗ trợ Python 3.8–3.11 và Java 8/11/17. Kafka 4.x yêu cầu Java 17 trở lên. Về quy mô mã nguồn, dự án có khoảng 43 900 dòng Python trong `src/`, 18 300 dòng kiểm thử, 2 700 dòng giao diện React và 1 800 dòng dashboard Streamlit.

3.3 Định danh và hợp đồng dữ liệu

3.3.1 Định danh tất định

Các định danh được tạo theo quy tắc tất định: cùng dữ liệu đầu vào sẽ cho cùng kết quả. Nhờ đó, việc thử lại (retry) hoặc phát lại dữ liệu không tạo thêm bản ghi trùng. Hệ thống sử dụng hàm băm

$$H(s) = \text{SHA-256}(s)[0{:}32], \qquad K(p_1, \ldots, p_n) = p_1 \,\|\, \texttt{\textbackslash x1f} \,\|\, p_2 \,\|\, \cdots \,\|\, p_n, \tag{11}$$

trong đó các thành phần được nối bằng ký tự phân cách `\x1f`, còn giá trị `null` được biểu diễn bằng `\x00`. Quy tắc này tránh việc hàm `concat_ws` của Spark bỏ qua `null`, khiến hai bộ giá trị khác nhau có cùng ID. Trước khi băm, các số thực được đưa về số nguyên theo hệ số cố định: vĩ độ và kinh độ nhân 10^7, hướng di chuyển (bearing) nhân 10^2, tốc độ nhân 10^3. Cách biểu diễn thống nhất giúp Python và Spark tạo ra cùng kết quả. Các định danh chính được liệt kê trong Bảng 7.

Bảng 7: Các định danh chính

Định danh	Cách tạo	Mục đích
`run_id`	uuid4, tạo một lần cho mỗi run	Tách Kafka key, checkpoint, raw, outbox và bảng phục vụ theo run
`gtfs_version`	SHA-256 của nội dung ZIP	Cố định một phiên bản lịch cho mỗi run
`scope_id`	SHA-256(route IDs đã sắp, route_type, scope policy, gtfs_version)[:16]	Phân biệt phạm vi xử lý
`capture_id`	`<vp|tu|al>-<UTC μs>Z-<sha256[:12]>`	Một lần thu thập HTTP
`snapshot_id`	`sn_` + H(run, scope, feed, header.ts, SHA-256 nội dung đã lọc)	Nhận diện snapshot logic; giữ ID khi thử lại
`generation_id`	`gn_` + H(run, feed, snapshot, pipeline_version)	Phiên bản dữ liệu phục vụ bất biến
`observation_id`	`ob_` + H(xe, descriptor, timestamp, vị trí lượng tử hóa, stop/status)	Thu thập lại không tạo lịch sử trùng
`prediction_id`	`pr_` + H(trip, start_date, start_time, route, direction, stop_sequence, stop_id, event_type)	Một dự báo của một lần ghé trạm
`source_order`	`%012d.%017d.%s` (feed ts, capture μs, snapshot_id)	Xác định thứ tự nguồn khi CAS pointer

Mỗi snapshot là một bản chụp nội dung của feed. Định danh `snapshot_id` không chứa `published_at`, `capture_id` hay UUID của lần thử lại, nên hai lần thu thập có cùng nội dung logic sẽ cho cùng định danh. Trường `source_order` có độ rộng cố định, giúp việc so sánh chuỗi tương đương với so sánh lần lượt thời điểm feed, thời điểm thu thập và ID. Con trỏ dữ liệu hiện tại (pointer) dựa vào thứ tự này để chọn snapshot mới hơn, thay cho thứ tự nhận dữ liệu hoặc thời điểm ghi vào Cassandra.

3.3.2 Envelope Kafka

Sau khi lọc theo phạm vi xử lý, mỗi snapshot được đóng gói thành một thông điệp JSON (envelope) theo schema version 1. Thông điệp gồm năm nhóm trường:

- phiên bản và phạm vi định danh: `run_id`, `scope_id`, `source_type`, `feed_type`, `gtfs_version`;
- thông tin truy vết nguồn gốc: `snapshot_id`, `capture_id`, `raw_path`, `raw_sha256`, `raw_bytes`;
- các mốc thời gian: `feed_timestamp`, `original_fetched_at`, `published_at`;
- số lượng bản ghi để đối chiếu: `source_entity_count`, `selected_entity_count`;
- nội dung dữ liệu: `header` và `entities`, được giữ nguyên dạng gốc.

Kích thước sau khi tuần tự hóa tối đa là 4 MiB. Nếu vượt giới hạn, dữ liệu gốc vẫn được lưu và hệ thống ghi mã `PAYLOAD_TOO_LARGE`. Thông điệp đó không được gửi sang Kafka, đồng thời nội dung cũng không bị cắt bớt.

3.4 Mô hình thời gian và đồng hồ

Các mốc thời gian được lưu riêng theo ý nghĩa của chúng, như trình bày trong Bảng 8. Cách tổ chức này dựa trên sự phân biệt giữa thời điểm sự kiện xảy ra (event time) và thời điểm hệ thống xử lý (processing time)

trong mô hình Dataflow [10].

Bảng 8: Các mốc thời gian và đồng hồ tương ứng

Trường	Ý nghĩa	Đồng hồ
`observation_time`	Thời điểm nguồn đo vị trí xe; có thể thiếu	Nguồn
`feed_timestamp`	Thời điểm header của feed; không chứng minh xe vừa được đo	Nguồn
`original_fetched_at`	Thời điểm thu thập gốc; giữ nguyên khi replay	Thực (lịch sử)
`published_at`	Thời điểm đưa vào Kafka trong run hiện tại	Thực
`view_published_at`	Thời điểm generation lần đầu được phép đọc	Thực
`application_clock`	Đồng hồ thực cho live, đồng hồ lịch sử cho replay/synthetic	Nghiệp vụ

Từ các mốc trên, hệ thống phân biệt độ trễ xử lý và tuổi dữ liệu nguồn:

$$\text{pipeline_visibility_latency} = \texttt{view_published_at} - \texttt{published_at}, \tag{12}$$

$$\text{source_observation_age} = \texttt{application_clock} - \texttt{observation_time}. \tag{13}$$

Đại lượng thứ nhất đo thời gian từ khi dữ liệu được gửi vào Kafka đến khi có thể đọc qua API, sử dụng đồng hồ thực (wall clock) của lần chạy hiện tại. Khoảng đo này không bao gồm thời gian chờ đến lần thu thập tiếp theo. Đại lượng thứ hai cho biết vị trí xe đã được ghi nhận cách thời điểm đánh giá bao lâu, nên phản ánh độ mới của dữ liệu nguồn.

Trong chế độ live, đồng hồ nghiệp vụ lấy giờ UTC hiện tại. Với replay và synthetic, thành phần phát dữ liệu (publisher) gửi sự kiện `clock_tick` trước mỗi mục dữ liệu và mỗi giây thực trong thời gian chờ. Mỗi sự kiện có `clock_sequence` tăng dần. Bộ xử lý (processor) chỉ chấp nhận sự kiện khi `clock_sequence` lớn hơn giá trị trước đó **và** `source_clock` không lùi; điều kiện được kiểm tra bằng LWT trên `run_metadata`. Khi tạm dừng phát lại, đồng hồ nghiệp vụ dừng theo, còn tín hiệu duy trì hoạt động (heartbeat) vẫn chạy theo đồng hồ thực. Giờ dự báo đến trạm không được dùng làm đồng hồ của hệ thống.

3.5 Thu thập dữ liệu và hàng đợi gửi bền vững

Bộ thu thập (collector) truy vấn độc lập ba feed theo chu kỳ 30 giây, mỗi feed sử dụng một HTTP client riêng. Thời gian chờ tối đa được đặt cho từng bước: kết nối 5 s, đọc 20 s, ghi 5 s và chờ lấy kết nối từ pool 5 s. Mỗi yêu cầu được thử tối đa 3 lần. Với lỗi tạm thời, khoảng chờ trước khi thử lại được tăng dần và ngẫu nhiên hóa theo công thức

$$\delta_k = \min(10,\ 1 \cdot 2^k) \cdot U(0{,}5;\ 1{,}5) \text{ giây}, \tag{14}$$

Collector cũng tuân theo header `Retry-After` với giới hạn 300 giây. Nếu bỏ lỡ một lượt thu thập, hệ thống chuyển sang lượt kế tiếp để tránh dồn yêu cầu. Một lần thu thập thành công được xử lý theo thứ tự sau:

1. **Lưu dữ liệu gốc trước khi gửi Kafka.** Nội dung phản hồi sau giải mã được nén gzip với `mtime=0` để tạo kết quả tất định, kèm mã SHA-256 và tệp metadata đi cùng. Dung lượng dành cho dữ liệu gốc là 2 GiB. Khi vượt giới hạn, collector dừng thay vì tự xóa dữ liệu.

2. **Chuẩn bị thông điệp.** Collector lọc sơ bộ theo các tuyến thuộc phạm vi xử lý, tính `snapshot_id` và tạo envelope.

3. **Lưu vào hàng đợi gửi.** Envelope được ghi vào outbox SQLite của lần chạy, với `journal_mode=WAL`, `synchronous=FULL` và trạng thái ban đầu là `pending`. Khóa `flock` ngăn hai collector chạy đồng thời trong cùng một run.

4. **Gửi sang Kafka.** Bộ phận chuyển tiếp (dispatcher) gửi thông điệp tới broker. Trạng thái `published` chỉ được ghi khi producer nhận xác nhận trong delivery callback, kèm partition và offset do broker trả về.

Quy trình trên áp dụng mẫu transactional outbox [12]. Nếu collector dừng đột ngột sau khi broker xác nhận nhưng trước khi cập nhật SQLite, thông điệp sẽ được gửi lại với cùng `capture_id` và `snapshot_id`. Các định danh tất định giúp tầng xử lý nhận biết lần gửi lặp này.

Nếu xảy ra lỗi như hết thời gian chờ, JSON hỏng, header sai hoặc `incrementality` không được hỗ trợ, collector giữ dữ liệu gốc và chuyển bản ghi lỗi vào hàng đợi DLQ. Hệ thống không tạo snapshot rỗng cho những trường hợp này, vì điều đó có thể khiến giao diện báo "0 xe" dù nguồn chỉ tạm thời gặp lỗi.

3.6 Vận chuyển qua Kafka

Kafka được triển khai với một broker ở chế độ KRaft, không dùng ZooKeeper, và có năm topic như Bảng 9. Mỗi topic có một partition với hệ số sao chép bằng 1. Thời gian lưu thông điệp trên Kafka là 6 giờ; dữ liệu cần lưu bền vững được giữ trong kho dữ liệu gốc và Cassandra [7].

Bảng 9: Các topic Kafka

Topic	Nội dung	Key
`mbta.vehicle_positions.snapshots.v1`	Snapshot vị trí xe đã lọc theo phạm vi	`run_id|vehicle_positions`
`mbta.trip_updates.snapshots.v1`	Snapshot dự báo chuyến	`run_id|trip_updates`
`mbta.alerts.snapshots.v1`	Snapshot cảnh báo	`run_id|alerts`
`mbta.control.v1`	Vòng đời run, clock tick, trạng thái collector/publisher	`run_id|control`
`mbta.dlq.v1`	Lỗi kèm mã lý do, tham chiếu và trích đoạn $\leq$ 2 KiB	`run_id|feed_type`

Producer được cấu hình `enable.idempotence=true` và `acks=all`. Việc nén được tắt bằng `compression.type=none` để giới hạn envelope 4 MiB phản ánh đúng kích thước truyền thực tế. Giới hạn `max.message.bytes` của topic được đặt là 5 MiB để có phần dự phòng. Khóa Kafka bắt đầu bằng `run_id`, giúp processor phân biệt thông điệp của các lần chạy dù cùng đọc một topic.

3.7 Xử lý luồng với Spark Structured Streaming

3.7.1 Nguồn Kafka và checkpoint

Processor sử dụng một truy vấn Spark Structured Streaming [5] để đọc đồng thời bốn topic, gồm ba feed dữ liệu và một topic điều khiển. Dữ liệu được xử lý theo các lô nhỏ (micro-batch) bằng `foreachBatch`, với chu kỳ kích hoạt 5 giây. Cấu hình nguồn trong `processing/stream_job.py` như sau:

```
def kafka_source_options(cfg, run_id: str) -> dict[str, str]:
    t = cfg.kafka.topics
    return {
        "kafka.bootstrap.servers": cfg.kafka.bootstrap_servers,
        "subscribe": ",".join([t.vehicle_positions, t.trip_updates, t.alerts, t.control]),
        "startingOffsets": "earliest",
        "failOnDataLoss": "true",
        "kafka.max.partition.fetch.bytes": str(cfg.spark.max_partition_fetch_bytes),
        "kafka.fetch.max.bytes": str(cfg.spark.fetch_max_bytes),
        "groupIdPrefix": f"tp-{run_id[:8]}",
        "includeHeaders": "false",
    }
```

Ba cấu hình sau quyết định cách hệ thống đọc và phục hồi dữ liệu:

- `startingOffsets=earliest` được dùng khi tạo checkpoint mới, kết hợp lọc theo khóa Kafka. Nhờ đó, các thông điệp phát trước khi truy vấn sẵn sàng vẫn được đọc. Cách này cũng khiến processor phải đọc qua dữ liệu của những run khác còn trong thời gian lưu.
- `failOnDataLoss=true` làm truy vấn dừng nếu offset trong checkpoint đã bị xóa do hết thời gian lưu. Hướng dẫn vận hành (runbook) mô tả cách phát lại dữ liệu vào một run mới khi gặp trường hợp này.
- `maxOffsetsPerTrigger` không được đặt vì giới hạn theo partition có thể tách một clock tick khỏi snapshot được phát ngay sau nó. Processor kiểm soát kích thước bằng cách báo lỗi khi một batch vượt `MAX_SNAPSHOTS_PER_BATCH = 3000`.

Phiên Spark sử dụng `spark.sql.session.timeZone=UTC` và `-Duser.timezone=UTC` để tránh sai lệch khi chuyển giờ lịch GTFS. Với cấu hình `spark.sql.ansi.enabled=false`, phép ép kiểu thất bại trả về `NULL`; bước phân tích dữ liệu dùng giá trị này để gắn mã lý do phù hợp. Spark UI được tắt bằng `spark.ui.enabled=false`.

3.7.2 Các bước trong một micro-batch

Trong mỗi micro-batch, `BatchProcessor` xử lý dữ liệu theo các bước:

1. **Tổng hợp sự kiện điều khiển:** cập nhật đồng hồ nghiệp vụ và trạng thái run (`CREATED` → `RUNNING` ⇄ `PAUSED` → `FINISHED`/`FAILED`) bằng LWT.
2. **Phân tích và kiểm tra envelope:** kiểm tra phiên bản schema, header, `incrementality` và `gtfs_version` của run. Thông điệp hỏng được đưa vào DLQ kèm mã lý do như `INVALID_JSON`, `STATIC_VERSION_MISMATCH`, ...

3. **Lập kế hoạch xử lý snapshot:** tạo hoặc đọc lại bản mô tả xử lý (manifest), xác định snapshot cha (parent), đồng thời nhận diện snapshot trùng hoặc đến muộn.

4. **Tạo generation:** tách các entity thành dòng dữ liệu, nối với bảng tham chiếu GTFS đúng phiên bản, tính ID và loại quan sát trùng theo `observation_id`. Tiếp theo, processor xác định lịch, tính độ trễ, chọn sự kiện dùng cho KPI và tổng hợp theo tuyến. Processor cũng đối chiếu các điều kiện áp dụng cảnh báo (selector) để tạo dữ liệu cảnh báo, đồng thời sinh các dòng chất lượng.

5. **Ghi và công bố dữ liệu:** ghi các bảng phục vụ, lịch sử và chất lượng, đối chiếu số dòng rồi công bố generation theo giao thức ở Mục 3.8.

6. **Lưu dữ liệu curated:** ghi lịch sử Parquet theo giao thức xác nhận hoàn tất (commit) từng batch ở Mục 3.10.

Các phép biến đổi dữ liệu được thực hiện bằng DataFrame API, tránh dùng vòng lặp Python trên tập dữ liệu lớn. Logic xử lý lấy thời gian từ đồng hồ nghiệp vụ đã cố định trong manifest, thay vì gọi `current_timestamp()`. Nhờ đó, kết quả không thay đổi chỉ vì một batch được xử lý lại ở thời điểm khác.

3.7.3 Xác định lịch và phiên chuyến

Để tính độ trễ, hệ thống cần xác định đúng ngày dịch vụ và lần ghé trạm của từng dự báo:

- Khi TripDescriptor có `start_date`, hệ thống kiểm tra chuyến có hoạt động trong ngày đó hay không dựa trên `calendar` và `calendar_dates`. Nếu không hoạt động, kết quả là `unresolved`.
- Khi thiếu `start_date`, hệ thống xét ba ngày ứng viên $L-1$, L, $L+1$, với L là ngày địa phương của `metrics_as_of`. Chỉ giữ các ngày có dịch vụ hoạt động và có đồng hồ nằm trong cửa sổ chuyến ± 12 giờ. Nếu còn đúng một ứng viên, kết quả là `resolved (inferred)`; nếu có nhiều ứng viên, kết quả là `ambiguous`.
- Trạm được đối chiếu bằng `stop_sequence`. Trường `stop_id` chỉ được dùng nếu trạm xuất hiện duy nhất trong chuyến. Với chuyến đi qua cùng trạm hai lần nhưng thiếu `stop_sequence`, hệ thống ghi `UNRESOLVED_STOP_SEQUENCE`.

Khi xác định được ngày dịch vụ, mỗi phiên chuyến (trip instance) có khóa `trip_id|YYYY-MM-DD`.

3.7.4 Tính KPI trên Spark

Hệ thống dùng hàm cửa sổ (window function) để chọn sự kiện đại diện (target) cho mỗi phiên chuyến. Ở mỗi lần ghé trạm, hệ thống chọn một sự kiện và ưu tiên arrival. Sau khi lọc các sự kiện tương lai, sự kiện có `stop_sequence` nhỏ nhất được chọn làm target. Đoạn mã dưới đây được trích từ `processing/kpi.py`:

```
per_stu = Window.partitionBy("_inst", "stu_pos").orderBy(
    F.when(F.col("event_type") == "arrival", 0).otherwise(1).asc()
)
tev = events.withColumn("_ev_rank", F.row_number().over(per_stu)).where(F.col("_ev_rank") == 1)
future = tev.where(
    (F.col("event_epoch") * F.lit(1_000_000) > F.lit(int(metrics_as_of_us)))
    & ~F.col("stu_rel").isin(*NON_TARGET_STOP_RELATIONSHIPS)
)
first = Window.partitionBy("_inst").orderBy(F.col("sort_seq").asc_nulls_last(), F.col("stu_pos").asc())
targets = (future.where(F.col("stop_resolved"))
           .withColumn("_t_rank", F.row_number().over(first))
           .where(F.col("_t_rank") == 1) ...)
```

Lý do loại trừ được xác định bằng chuỗi `F.when` theo thứ tự `KPI_EXCLUSION_ORDER`, nên mỗi chuyến nhận tối đa một lý do. Kết quả được tổng hợp từ danh sách tuyến trong phạm vi xử lý bằng `left join`. Vì vậy, tuyến chưa có chuyến vẫn xuất hiện với số đếm bằng 0 và tỷ lệ `null`; mẫu số bằng 0 không bị hiển thị thành tỷ lệ 0%.

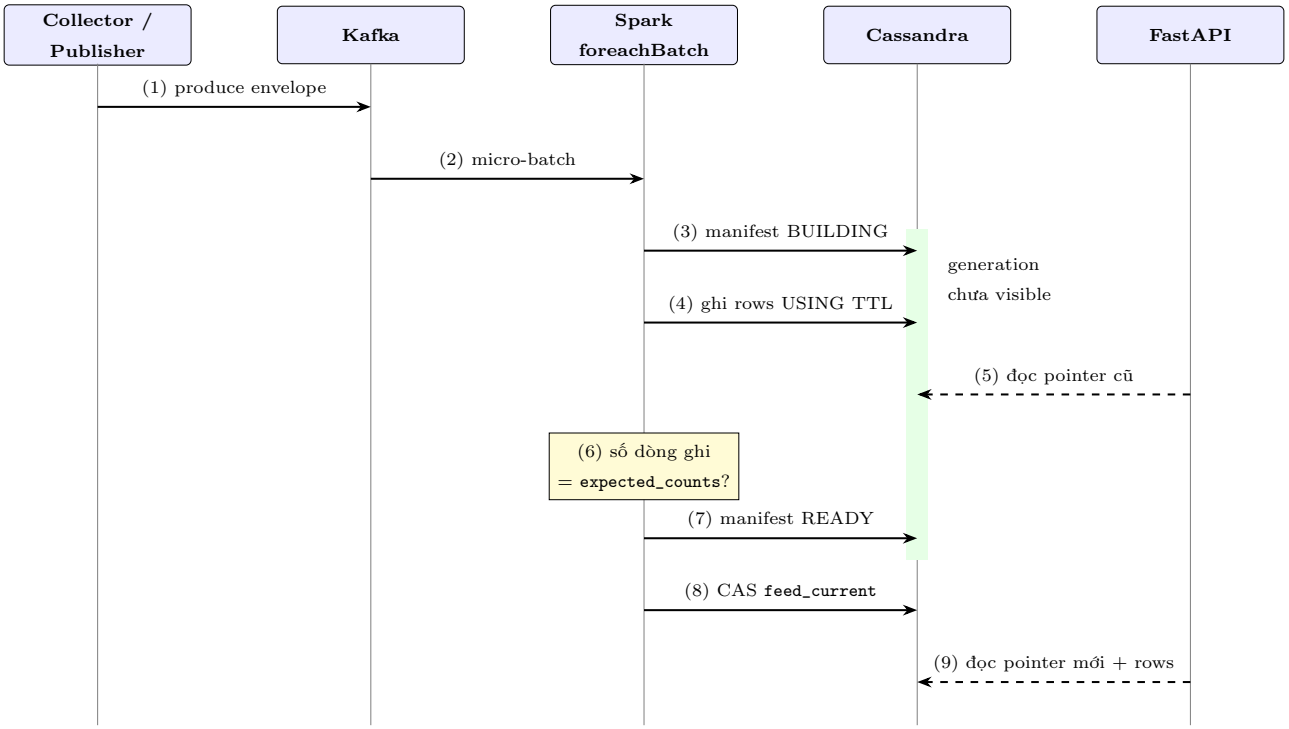
3.8 Giao thức công bố generation

3.8.1 Vấn đề

Một snapshot có thể tạo ra hàng trăm đến hàng nghìn dòng trên nhiều bảng Cassandra. Tuy nhiên, Cassandra không hỗ trợ giao dịch trên nhiều partition [8], còn logged batch nhiều partition cũng không bảo đảm tính cô lập khi đọc. Nếu API đọc dữ liệu trong lúc processor đang ghi, kết quả có thể chỉ chứa một phần số xe. Trường hợp processor dừng đột ngột còn khiến Spark phải xử lý lại micro-batch từ cùng offset.

3.8.2 Giải pháp: generation bất biến, manifest và pointer CAS

Để xử lý vấn đề này, TransitPulse kiểm soát **thời điểm dữ liệu được phép đọc**. Mỗi snapshot tạo một *generation*, tức một bộ dòng phục vụ bất biến có khóa partition chứa `snapshot_id`. API chỉ đọc generation đó sau khi con trỏ `feed_current(run_id, feed_type)` được cập nhật bằng phép so sánh và thay thế (compare-and-set, CAS). Thao tác CAS sử dụng giao dịch nhẹ (lightweight transaction, LWT) của Cassandra, dựa trên giao thức Paxos [9]. Trình tự được minh họa trong Hình 2.



Hình 2: *Trình tự công bố một generation. Trong các bước (3)–(7), API vẫn đọc generation cũ thông qua con trỏ hiện tại.*

Giao thức gồm các quy tắc sau:

1. **Cố định tham số từ lần tạo đầu tiên.** Manifest ở trạng thái `BUILDING` được tạo bằng `INSERT ... IF NOT EXISTS`. Khi thử lại, nếu dòng đã tồn tại, processor dùng các giá trị đã lưu gồm `parent_snapshot_id`, `metrics_as_of`, `expires_at`, `history_expires_at` và `built_as_late`. Kết quả logic vì vậy không phụ thuộc giờ thực của lần thử lại.

2. **Chọn snapshot cha.** Parent là snapshot mới hơn giữa hai ứng viên: snapshot mà pointer hiện tại trỏ tới và snapshot không trễ trước snapshot đang xử lý trong cùng feed, cùng micro-batch. Dữ liệu cha chỉ được kế thừa trong các trường hợp được kiểm soát, gồm vị trí khi có `OBSERVATION_TIME_REGRESSION` và cảnh báo khi snapshot chỉ hợp lệ một phần. Xe vắng mặt trong snapshot mới không được đưa từ parent sang danh sách fresh.

3. **Ghi dữ liệu theo cách idempotent.** Mỗi dòng có khóa xác định, nên ghi lại cùng một dòng không làm thay đổi kết quả logic. Hệ thống không dùng counter. TTL của mỗi lần ghi bằng số giây còn lại đến `expires_at` đối với bảng phục vụ (1 giờ), hoặc `history_expires_at` đối với lịch sử và chất lượng (3 ngày).

4. **Chuyển READY khi đủ số dòng.** Số dòng executor ghi vào từng bảng phải khớp `expected_counts` trong manifest. Nếu không khớp, batch thất bại, manifest chưa được chuyển sang READY và pointer giữ nguyên.

5. **Cập nhật CAS theo thứ tự nguồn.** Pointer chỉ tiến khi `source_order` mới lớn hơn giá trị hiện tại.

Nếu đã trỏ đúng snapshot, thao tác được xem là thành công idempotent. Nếu pointer đã trỏ tới snapshot mới hơn, snapshot đang xử lý được xem là đến muộn (*late*). Nó vẫn được lưu để kiểm tra và tra cứu lịch sử, kèm mã `LATE_SNAPSHOT_NOT_PUBLISHED`, còn pointer không bị lùi.

6. **Ghi độ trễ công bố một lần.** LWT `IF publish_outcome = null` bảo đảm visibility latency chỉ được ghi lần đầu. Các lần gửi lặp có chỉ số riêng và không tạo độ trễ âm.

7. **Dừng batch khi không ghi được hạ tầng bắt buộc.** Nếu Cassandra không ghi được, ngoại lệ được truyền ra ngoài để micro-batch thất bại và checkpoint chưa được commit. Khi khởi động lại với cùng checkpoint, Spark xử lý lại đúng `batch_id` và các offset đó.

Đoạn mã dưới đây mô tả vòng CAS trong `processing/batch.py`, đã rút gọn phần ghi log và chỉ số:

```
for attempt in range(1, attempts + 1):
    ptr = read_pointer(ctx.session, ctx.run_id, feed, serial=True)
    if ptr is not None and ptr["snapshot_id"] == sid:
        ...  # pointer already at this generation: idempotent success
        return RESULT_REUSED, applied
    if ptr is not None and str(ptr["source_order"]) > str(man["source_order"]):
        self._audit_quality(man, "LATE_SNAPSHOT_NOT_PUBLISHED", details={...})
        return RESULT_LATE, set_outcome_once(ctx.session, key, OUTCOME_LATE)
    now = ms_floor(utcnow())
    if remaining_s(man["expires_at"], now) < min_ttl:
        return RESULT_EXPIRED, ...
    row = pointer_row(man, now)
    if ptr is None:
        applied_ptr, _existing = bootstrap_pointer(ctx.session, row)   # INSERT ... IF NOT EXISTS
    else:
        applied_ptr = cas_pointer(ctx.session, row, ptr["snapshot_id"])  # UPDATE ... IF snapshot_id = ?
    if applied_ptr:
        lat = latency_ms(now, man.get("published_at"))
        applied = set_outcome_once(ctx.session, key, OUTCOME_PUBLISHED, now, lat)
        return RESULT_PUBLISHED, applied
raise PointerContention(f"pointer CAS for {feed}/{sid} failed {attempts} times")
```

Câu lệnh CAS tương ứng trong `storage/pointer.py`:

```
UPDATE feed_current SET snapshot_id = ?, generation_id = ?, source_order = ?, ...
WHERE run_id = ? AND feed_type = ? IF snapshot_id = ?
```

Với giao thức này, hệ thống cung cấp **giao nhận at-least-once kết hợp ghi và công bố idempotent**: một thông điệp có thể được xử lý nhiều lần nhưng kết quả logic không bị nhân đôi. Thiết kế chưa cung cấp exactly-once xuyên Kafka, Parquet và Cassandra [11]. Bảng 10 trình bày hành vi mong đợi trong các tình huống lỗi điển hình.

Bảng 10: Các kịch bản phục hồi và hành vi mong đợi

Tình huống	Hành vi	Kiểm thử
Cùng snapshot được gửi lại	Dùng lại manifest READY; số đếm và lịch sử không tăng	IT02
Snapshot cũ đến sau snapshot mới	Được đánh dấu READY nhưng không trở thành dữ liệu hiện tại; pointer và tọa độ không lùi	IT03
Dừng đột ngột sau khi ghi một phần, trước READY	API tiếp tục đọc generation cũ; lần thử lại hoàn tất với cùng ID và parent	IT10
Dừng đột ngột sau CAS, trước commit checkpoint	Lần thử lại nhận biết pointer đã đúng; giữ nguyên `view_published_at`, `expires_at`	IT11
Cassandra không ghi được	Batch thất bại, offset chưa commit; dữ liệu tồn được xử lý sau khi phục hồi	IT12
Generation hết TTL	API trả 503 `generation_expired`; snapshot rỗng hợp lệ trả 200 cùng `[]`	IT15

Processor hỗ trợ chủ động tạo lỗi (fault injection) để kiểm thử khả năng phục hồi. Mỗi kịch bản được kích hoạt bằng một tệp dùng một lần tại `faults/<run_id>/<point>.json`. Bốn điểm tạo lỗi gồm `after_partial_write`, `after_ready_before_cas`, `after_cas_before_commit` và `executor_write_error`. Khi gặp điểm đã chọn, processor thoát với mã 86, giúp phân biệt với mã 137 khi tiến trình bị dừng do thiếu bộ nhớ (OOM). Cơ chế này chỉ được bật khi chạy với cờ `--enable-fault-injection`.

3.9 Mô hình dữ liệu Cassandra

3.9.1 Nguyên tắc thiết kế

Schema trong `storage/migrations/0001_initial.cql` gồm 17 bảng. Mô hình được thiết kế từ nhu cầu truy vấn (**query-first**) và sử dụng phi chuẩn hóa, nghĩa là mỗi bảng được tổ chức để phục vụ trực tiếp một mẫu truy vấn. Các nguyên tắc chính gồm:

- truy vấn đọc theo partition và có `LIMIT`, tránh quét toàn bảng và không dùng `ALLOW FILTERING`;
- không dùng counter vì việc thử lại có thể làm tăng số đếm, đồng thời không dùng logged batch trên nhiều partition;
- dữ liệu điều khiển trong `feed_current`, `snapshot_manifest` và đồng hồ của `run_metadata` chỉ được ghi bằng LWT;
- bảng phục vụ dùng `USING TTL` dựa trên `expires_at`; bảng lịch sử và chất lượng dùng `history_expires_at`.

3.9.2 Các bảng chính

Bảng 11 liệt kê các bảng chính cùng khóa và mục đích sử dụng. Trong bảng, ký hiệu $((a, b), c)$ biểu diễn khóa phân vùng (partition key) (a, b) và cột sắp xếp trong phân vùng (clustering column) c.

Bảng 11: Bảng Cassandra, khóa, truy vấn sử dụng và TTL

Bảng	Khóa	Phục vụ	TTL
`run_metadata`	((run_id))	phạm vi, đồng hồ, chính sách của run	không
`feed_current`	((run_id, feed_type))	pointer; đọc một lần mỗi feed trong một yêu cầu	không
`snapshot_manifest`	((run_id, feed_type), snapshot_id)	READY, expected_counts, nguồn gốc dữ liệu	không
`routes_by_version`	((gtfs_version), route_id)	`/routes`	không
`stop_times_by_trip`	((gtfs_version, trip_id), stop_sequence)	`/trips/{id}/stops`	không
`geometries_by_route`	((gtfs_version, route_id), shape_id)	`/routes/{id}/geometry`	không
`vehicles_by_route_snapshot`	((run_id, snapshot_id, route_id), vehicle_id)	`/vehicles`, chỉ số	1 h
`predictions_by_stop_snapshot`	((run_id, snapshot_id, stop_id), predicted_at, prediction_id)	`/stops/{id}/predictions`	1 h
`route_metrics_by_snapshot`	((run_id, feed_type, snapshot_id), route_id)	`/routes/{id}/metrics`	1 h
`alerts_by_route_snapshot`	((run_id, snapshot_id, route_id), alert_id)	`/alerts`	1 h
`vehicle_observations_by_day`	((run_id, vehicle_id, utc_date), event_time DESC, observation_id)	`/vehicles/{id}/history`	3 ngày
`quality_by_snapshot`	((run_id, feed_type, snapshot_id), reason_code)	`/quality`	3 ngày
`pipeline_status`	((run_id), component)	`/pipeline/status`	không

Việc đưa `snapshot_id` vào khóa phân vùng giúp tách riêng dữ liệu của từng generation. Generation mới vì vậy không ghi đè lên dữ liệu đang được đọc. Các generation cũ được dọn theo TTL, giảm nhu cầu xóa chủ động bằng `DELETE` và các tombstone do thao tác này tạo ra. Dưới đây là định nghĩa rút gọn của bảng vị trí xe:

```
CREATE TABLE IF NOT EXISTS transitpulse.vehicles_by_route_snapshot (
  run_id text, snapshot_id text, route_id text, vehicle_id text,
  observation_id text, record_kind text,
  observation_time timestamp, event_time timestamp, event_time_origin text,
  freshness_known boolean, position_valid boolean, invalid_reason text,
  latitude double, longitude double, bearing double, speed_mps double,
  trip_id text, direction_id int, start_date text, start_time text,
  is_inherited boolean, inherited_from_snapshot_id text,
  quality_flags set<text>,
  PRIMARY KEY ((run_id, snapshot_id, route_id), vehicle_id)
);
```

Bảng lịch sử `vehicle_observations_by_day` được phân vùng theo bộ (run, xe, ngày UTC) để giới hạn kích thước từng partition. Dữ liệu bên trong được sắp giảm dần theo `event_time`, giúp truy vấn "một giờ gần nhất" chỉ cần đọc phần đầu phân vùng.

3.9.3 Nhất quán khi đọc

Với mỗi yêu cầu, API đọc pointer **một lần** cho từng feed và dùng cùng `snapshot_id` cho các truy vấn còn lại. Khi generation còn hạn nhưng số dòng đọc được không khớp `expected_counts["<bảng>/<khóa>"]`, API trả 503 với mã `generation_inconsistent`. Quy tắc này áp dụng cả khi thiếu dòng lẫn khi thừa dòng từ một lần tạo generation bị hủy.

Nếu generation đã hết hạn, API trả 503 với mã `generation_expired`. Chỉ snapshot rỗng hợp lệ mới được trả 200 cùng danh sách rỗng, giúp người dùng phân biệt trạng thái "0 xe" với lỗi dữ liệu. Cấu hình demo dùng `LOCAL_ONE` cho đọc và ghi thông thường, còn LWT dùng `SERIAL`, phù hợp với triển khai một node RF1.

3.10 Lưu trữ lịch sử và báo cáo batch

Bên cạnh Cassandra, mỗi micro-batch lưu dữ liệu lịch sử đã xử lý ở dạng Parquet theo một giao thức commit riêng. Kết quả của từng snapshot, gọi là *unit*, được ghi vào thư mục `_staging/batch_id=N/` rồi đánh dấu hoàn tất. Processor tiếp tục dùng `os.rename` để chuyển cả thư mục sang `batch_id=N/`; đây là thao tác nguyên tử khi thực hiện trên cùng hệ thống tệp. Cuối cùng, tệp `_COMMITTED` được tạo để liệt kê chính xác các tệp đầu ra.

Thành phần đọc chỉ sử dụng batch có `_COMMITTED` và đọc đúng những tệp được liệt kê trong đó. Cách này giúp dữ liệu ghi dở chưa được đưa vào báo cáo.

Tác vụ `processing/report_job.py` (FR09) dùng Spark batch để phân tích các batch đã commit, gồm:

- hoạt động xe theo tuyến trong các cửa sổ thời gian sự kiện dài 1 phút và 5 phút (`F.window`), bao gồm số xe phân biệt và số quan sát sau khi loại trùng theo `observation_id`. Tốc độ được suy ra từ hai quan sát liên tiếp bằng công thức haversine, đồng thời loại các bước nhảy GPS có tốc độ trên 40 m/s;
- tỷ lệ chuyến trễ dự kiến theo tuyến trong cửa sổ 5 phút. Dữ liệu được loại trùng theo (`snapshot_id`, `prediction_id`) và lấy generation mới nhất của từng phiên chuyến;
- các phân vị p50/p95 của độ trễ công bố theo feed, kèm số mẫu dùng để tính.

Định danh `report_id` được tạo từ hàm băm của danh sách batch đầu vào. Vì vậy, chạy lại với cùng dữ liệu đầu vào không tạo thêm báo cáo.

3.11 Tầng phục vụ FastAPI

FastAPI phục vụ dữ liệu theo chế độ chỉ đọc. Mỗi phản hồi có dạng `{data, meta, warnings}`. Với từng feed, trường `meta` cung cấp `snapshot_id`, `feed_timestamp`, `metrics_as_of`, tuổi dữ liệu nguồn, ngưỡng xác định

dữ liệu cũ (stale) và thời điểm hết hạn. Phản hồi lỗi gồm mã lỗi, thông điệp và gợi ý xử lý. Các endpoint chính được liệt kê trong Bảng 12.

Bảng 12: Các endpoint chính và trình tự đọc Cassandra

Endpoint	Trình tự đọc
`GET /health/live`, `/health/ready`	Không đọc; `system.local` + `schema_migrations`
`GET /runs`, `/runs/{run_id}`	`runs_by_project` (LIMIT) → `run_metadata`, 3 pointer, `pipeline_status`
`GET /routes`, `/routes/{id}/stops|geometry`	`run_metadata` → bảng tĩnh theo `gtfs_version` của run
`GET /vehicles`	`feed_current(VP)` → manifest → `vehicles_by_route_snapshot` theo từng tuyến ($\leq$ 2000 dòng)
`GET /vehicles/{id}/history`	`vehicle_observations_by_day` theo từng phân vùng ngày UTC ($\leq$ 24 h, phân trang)
`GET /stops/{id}/predictions`	`feed_current(TU)` → manifest → `predictions_by_stop_snapshot` (`predicted_at` $\geq$ clock − 60 s)
`GET /routes/{id}/metrics`	`feed_current(VP, TU)` → `route_metrics_by_snapshot` + xe (tính lại độ mới)
`GET /alerts`	`feed_current(AL)` → `alerts_by_route_snapshot` (đọc từng tuyến, loại trùng `alert_id`)
`GET /quality`, `/pipeline/status`	3 pointer → `quality_by_snapshot`, `pipeline_status`

Tầng phục vụ được tổ chức theo bốn nguyên tắc:

- **Tính trạng thái tại thời điểm yêu cầu.** Độ mới của vị trí xe, trạng thái hiệu lực của cảnh báo (ALWAYS, ACTIVE, UPCOMING, EXPIRED) và mức độ cũ của feed đều được tính theo đồng hồ nghiệp vụ tại thời điểm API nhận yêu cầu.
- **Chỉ lưu đệm dữ liệu bất biến.** API lưu đệm generation READY và dữ liệu tĩnh, với 256 mục và TTL tối đa 30 s. Pointer, đồng hồ và độ mới của dữ liệu được đọc hoặc tính lại.
- **Giới hạn tài nguyên sử dụng.** API chạy một worker uvicorn, giới hạn thời gian mỗi câu lệnh là 2 s và mỗi yêu cầu là 10 s. Trang kết quả mặc định có 100 dòng, tối đa 500 dòng; token phân trang được ký HMAC.
- **Phân biệt nguồn cảnh báo.** Cảnh báo dịch vụ MBTA và cảnh báo chất lượng pipeline thuộc hai nhóm riêng. Nhờ đó, lỗi của pipeline không bị hiểu nhầm là thông báo từ MBTA.

3.12 Giao diện người dùng

Hệ thống cung cấp ba giao diện, cùng lấy dữ liệu qua HTTP từ API:

- **Console React** sử dụng Vite, Tailwind và MapLibre, gồm sáu màn hình: tổng quan, bản đồ, trạm, cảnh báo, chất lượng và lịch sử xe. Bản đồ sử dụng nền CARTO trên dữ liệu OpenStreetMap, nên trình duyệt cần có kết nối mạng.
- **Dashboard Streamlit + PyDeck** phục vụ việc theo dõi khi chạy trên máy cá nhân.

- **Giao diện notebook** sử dụng ipywidgets, gồm năm tab trên Colab. Tab bản đồ tự cập nhật vị trí xe mỗi 5 giây.

Các phép tính nghiệp vụ được thực hiện ở phía xử lý và API. Giao diện không tự thay dữ liệu thiếu bằng 0; khi API trả 503, giao diện hiển thị trạng thái "unavailable" để thể hiện dữ liệu chưa sẵn sàng.

3.13 Replay và dữ liệu synthetic

Một bản ghi để phát lại (recording) gồm nội dung gốc, dòng thời gian, manifest có checksum và `gtfs_version` được dùng khi thu thập. Trước khi phát lại, hệ thống kiểm tra checksum và phiên bản GTFS để dữ liệu lịch sử được nối đúng với lịch tương ứng. Recording cũ không được ghép với tệp ZIP mới tải.

Publisher phát dữ liệu theo dòng thời gian ở tốc độ 1x, 10x hoặc 50x. Việc điều khiển được thực hiện qua `publisher_control.json`, còn trạng thái xác nhận được ghi vào `publisher_state.json`. Các thao tác hỗ trợ gồm tạm dừng (pause), tiếp tục (resume) và phát từng bước (step).

Dữ liệu synthetic được sinh tất định từ seed, đóng gói thành recording và phát bằng cùng publisher. Ba loại dữ liệu gồm *golden* (bộ dữ liệu kiểm thử viết tay), *demo* và *stress*. Loại stress nhân bản các chuyến tĩnh của 5 tuyến để tạo tải 100–1000 bản ghi chuẩn hóa mỗi giây.

3.14 Vận hành, triển khai và quan sát

3.14.1 Môi trường chạy trực tiếp

Để chạy trên Google Colab, nơi không có Docker, systemd hay sudo, dự án xây dựng một môi trường điều phối bằng Python cho các dịch vụ chạy trực tiếp:

- **Bootstrap** (`scripts/bootstrap_colab.sh`) tải JDK, Kafka, Cassandra, Node.js, cloudflared và connector Spark theo `deploy/runtime-manifest.json`. Mỗi gói được đối chiếu với tệp checksum chính thức (SHA-256 hoặc SHA-512) trước khi giải nén an toàn. Kafka và Cassandra có địa chỉ tải dự phòng tại `archive.apache.org`.
- **Doctor** kiểm tra điều kiện trước khi chạy, gồm kiến trúc và số CPU, RAM khả dụng, dung lượng đĩa, inode, cổng dịch vụ, phiên bản Java/Python và mã băm cấu hình.
- **Supervisor** khởi động và dừng Kafka, Cassandra, processor, collector/publisher, API cùng giao diện. Thành phần này ghi nhận quyền sở hữu từng tiến trình để dừng đúng tiến trình cần quản lý, không dùng `pkill`/`killall`. Việc dừng dịch vụ không xóa dữ liệu.
- Ba cấu hình triển khai dùng chung mã nguồn, schema và API: `colab-native`, `local-native` và `local-docker` (Docker Compose). Trong đó, `colab-native` là cấu hình đã được chạy xuyên suốt từ đầu vào đến đầu ra.

Cấu hình `colab-native` dành 1 GiB heap cho Kafka, 4 GiB heap và 2 GiB direct memory cho Cassandra, cùng 16 GiB cho Spark driver ở chế độ `local[2]`. Tổng ngân sách khoảng 23 GiB trên máy ảo có RAM cao. Trong demo và golden smoke trên Colab, tính năng whole-stage codegen của Spark được tắt vì việc biên dịch kế hoạch xử lý dự báo mất nhiều phút ở mỗi micro-batch, vượt thời gian dành cho golden smoke. Khi đo thông lượng, benchmark bật lại codegen thông qua biến môi trường `TP_SPARK_WHOLE_STAGE`.

3.14.2 Truy cập và an toàn

Các cổng dịch vụ chỉ lắng nghe tại `127.0.0.1`, còn CORS giới hạn nguồn truy cập bằng danh sách cho phép. Trên Colab, notebook có thể đưa console React ra Internet qua Cloudflare quick tunnel, với URL ngẫu nhiên dạng `*.trycloudflare.com`. Tệp thực thi `cloudflared` được kiểm tra theo mã SHA-256 đã cố định.

Đường dẫn tunnel là **công khai và không có xác thực**. Console chỉ phục vụ dữ liệu công khai MBTA hoặc dữ liệu synthetic; Kafka và Cassandra vẫn không được mở ra ngoài. Có thể tắt tunnel bằng `ENABLE_CLOUDFLARE_TUNNEL = False`. Phiên bản Spark 3.5.7 có lỗ hổng CVE-2026-32773 liên quan đến XSS ở Spark History Server. Trong cấu hình dự án, Spark UI được tắt và History Server không được chạy.

3.14.3 Quan sát

Mỗi thành phần ghi log JSON có cấu trúc, gồm giai đoạn xử lý, trạng thái, batch, feed, snapshot và thời lượng. Các chỉ số Prometheus được xuất dưới dạng textfile, bao gồm số generation theo kết quả công bố (`published`, `reused`, `late`, `expired`), phân bố độ trễ công bố theo feed và lượng dữ liệu Kafka chưa xử lý theo topic, partition.

Thành phần `StreamingQueryListener` ghi tiến độ vào `query_progress.jsonl` và cập nhật heartbeat trong `pipeline_status`. Để kiểm tra nguồn gốc dữ liệu, lệnh `demo.py lineage` lần ngược một snapshot qua pointer, manifest, dữ liệu gốc, Kafka, batch và API. Trong quá trình này, lệnh đối chiếu SHA-256 của dữ liệu gốc, đọc lại thông điệp Kafka tại offset đã ghi và báo kết quả kiểm tra tính nhất quán.

4 Kiểm chứng và đánh giá thực nghiệm

4.1 Chiến lược kiểm chứng

Việc đánh giá dự án dựa trên kết quả kiểm thử và số liệu đo được. Một bước chỉ được ghi `PASS` khi có bằng chứng xác nhận; bước chưa chạy được ghi `NOT_RUN` kèm lý do. Tương tự, bộ công cụ benchmark giữ giá trị `null` cho chỉ số chưa đo được và ghi rõ nguyên nhân. Các mục tiêu thiết kế, chẳng hạn p95 < 10 s, được trình bày riêng để người đọc có thể đối chiếu với kết quả thực tế.

4.1.1 Các tầng kiểm thử

Bảng 13 trình bày phạm vi và hạ tầng của từng tầng kiểm thử. Số hàm kiểm thử trong bảng được đếm trực tiếp từ mã nguồn. Khi chạy, số trường hợp kiểm thử lớn hơn số hàm vì một hàm có thể được chạy với nhiều bộ tham số thông qua `pytest.mark.parametrize`.

Bảng 13: Các tầng kiểm thử

Tầng	Số tệp	Hàm test	Phạm vi	Hạ tầng
Unit	38	438	ID, quy tắc xử lý, lịch GTFS/DST, lịch dịch vụ, phạm vi, outbox, HTTP giả lập, API với repository giả, replay, oracle	Python thuần
Runtime	18	133	Checksum, giải nén an toàn, supervisor/registry, phiên chạy, reset, doctor, tunnel, xuất dữ liệu	Python thuần
Spark	12	117	Phân tích dữ liệu, vị trí xe, dự báo, KPI, cảnh báo, ghi và công bố dữ liệu, curated, truy vết trên golden fixture	JDK + PySpark `local[1]`
Integration	10	18	IT01–IT16 với Kafka, Cassandra, Spark thực tế qua native runner	Toàn hệ thống
Golden smoke	–	5 bước	Golden fixture qua Kafka → Spark → Cassandra → API	Toàn hệ thống

4.1.2 Golden fixture và oracle độc lập

Golden fixture tại `tests/fixtures/golden/` là bộ dữ liệu kiểm thử **được tạo thủ công, không phải dữ liệu MBTA**. Bộ dữ liệu này gồm GTFS tĩnh với đơn vị vận hành `SYN`, hai tuyến `R_A`, `R_B`, các trạm từ `S1` đến `S9` và ba feed VP/TU/Alerts có cấu trúc tương tự JSON của MBTA. Các giá trị kỳ vọng trong `expected.json` đều được tính bằng tay từ đặc tả thiết kế, mỗi giá trị được tính độc lập hai lần. Nhờ đó, kết quả kỳ vọng không phụ thuộc vào kết quả do hệ thống tạo ra. Bảng thời gian epoch trong `PROVENANCE.md` cũng được kiểm tra bằng test.

Dự án còn dùng một **oracle độc lập** tại `tests/oracle/` để tính lại kết quả bằng thư viện chuẩn Python. Test `test_oracle_isolation.py` phân tích cây cú pháp AST và chạy tiến trình con để kiểm tra rằng oracle không import `transitpulse` hoặc `pyspark`. Chuỗi đối chiếu gồm oracle(fixture) = `expected.json` và pipeline (Spark, API, Cassandra) = `expected.json`. Cách kiểm chứng qua ba nguồn độc lập giúp giảm khả năng phần

xử lý và kết quả kỳ vọng cùng mắc một lỗi.

4.1.3 Kịch bản golden

Kịch bản golden gồm năm bước. Sau mỗi bước, bộ phát dữ liệu (publisher) và bộ xử lý (processor) chờ nhau tại một điểm đồng bộ (barrier) trước khi tiếp tục. Bảng 14 trình bày các giá trị kỳ vọng chính của tuyến `R_A`.

Bảng 14: Năm bước của kịch bản golden và giá trị kỳ vọng chính (tuyến R_A)

Bước	Đầu vào (đồng hồ)	Kỳ vọng
S1	VP1, TU1, ALERTS1 (12:00:30Z)	Ba con trỏ được tạo. Xe: fresh 1, stale 1, unknown 1, membership 3. KPI: 2 trip instance, eligible 2, late 1, rate 0,5, mean 210 s. Tuyến R_B không có chuyến: rate và mean là `null`.
S2	VP2 (12:01:00Z)	Con trỏ VP chuyển sang VP2. Xe V2 vắng trong VP2 nên không còn được tính: fresh 1, unknown 1, membership 2. Lịch sử V1 và V3 mỗi xe có 2 dòng.
S3	Phát lại VP2 (12:01:00Z)	VP2 được phát lại khi đang là snapshot hiện hành: không tạo generation mới, con trỏ giữ nguyên, lịch sử không tăng, bộ đếm duplicate VP = 1.
S4	Phát lại VP1 (12:01:00Z)	VP1 vừa trùng vừa đến muộn: con trỏ **không lùi** về VP1, lịch sử không đổi, duplicate VP = 2, late VP = 1, có bản ghi audit `LATE_SNAPSHOT_NOT_PUBLISHED`.
S5	Chỉ tiến đồng hồ (12:03:10Z)	Không có dữ liệu mới nhưng trạng thái thay đổi theo đồng hồ: feed VP tuổi 130 s (stale), TU tuổi 170 s (stale, ngưỡng 90 s), Alerts tuổi 170 s (chưa stale, ngưỡng 300 s). Xe: fresh 0, stale 1, unknown 1. KPI giữ giá trị của generation nhưng được đánh dấu stale.

4.1.4 Kịch bản tích hợp

Bảng 15 liệt kê 16 kịch bản kiểm thử tích hợp với Kafka, Cassandra và Spark thực tế. Hai kịch bản gây gián đoạn môi trường, gồm IT12 dừng Cassandra và IT16 tắt mạng, được bố trí chạy cuối.

Bảng 15: Kịch bản tích hợp IT01–IT16

Mã	Kịch bản	Điều kiện đạt
IT01	Golden fixture ban đầu	Số đếm, tỉ lệ 0,5 và trung bình 210 s khớp tại API và nơi ghi dữ liệu
IT02	Snapshot mới phát hai lần	Membership không tăng; lịch sử không lặp cùng quan sát
IT03	Snapshot cũ tới sau snapshot mới	Con trỏ và vị trí không lùi; snapshot đến muộn có audit
IT04	Đồng hồ tiến, không có vị trí mới	Xe chuyển stale; dự báo cũ báo stale
IT05	Snapshot rỗng hợp lệ và dữ liệu sai định dạng/timeout	Lỗi nguồn không biến thành snapshot rỗng
IT06	Thiếu timestamp, GPS không hợp lệ	Unknown không được tính là fresh; cách xử lý và số đếm đúng
IT07	Ngày dịch vụ không rõ, trạm lặp	Không tự đoán lịch hoặc trạm; ghi lý do khi loại khỏi KPI
IT08	Uncertainty bus, chuyến bị hủy	Các giá trị 300, 301, >301 và trường hợp thiếu được xử lý đúng quy tắc
IT09	Cảnh báo cập nhật một phần	Không đóng hàng loạt cảnh báo cũ; thông tin truy vết được kế thừa đúng
IT10	Dừng sau khi ghi một phần	API giữ generation cũ; khởi động lại hoàn tất đúng generation
IT11	Dừng sau CAS, trước commit checkpoint	Dùng lại generation READY; thời điểm công bố đầu tiên và TTL không đổi
IT12	Cassandra dừng	Batch thất bại; offset không được coi là đã xử lý
IT13	Message gần 4 MiB và vượt ngưỡng	Kiểm tra đúng giới hạn kích thước; thông điệp vượt ngưỡng vào DLQ
IT14	Cô lập run, tạm dừng replay	Kết quả tách theo run; đồng hồ nghiệp vụ dừng, heartbeat theo giờ máy tiếp tục
IT15	Generation hết TTL	503 `generation_expired`, phân biệt với 0 dòng hợp lệ
IT16	Demo khi tắt mạng	Pipeline, API và giao diện lõi vẫn dùng được

Mỗi lần push hoặc tạo pull request, CI trên GitHub Actions chạy Ruff, các kiểm thử unit và runtime, đồng thời kiểm tra tĩnh notebook, cấu hình, fixture và migration. Các kiểm thử tích hợp được chạy qua một workflow kích hoạt thủ công.

4.2 Môi trường thực nghiệm

Số liệu thực nghiệm được lấy từ hai lần chạy trên Google Colab với cùng một loại máy ảo. Cấu hình môi trường được trình bày trong Bảng 16. Hai lần chạy gồm:

- **Demo (Run All)** ngày 19/10/2026, tại commit `4d6669e`: dựng môi trường, chạy kiểm thử, golden smoke, thu thập trực tiếp trên 10 tuyến và truy vết dữ liệu. Số liệu được lấy từ kết quả đã lưu trong notebook `TransitPulse_MBTA_Colab_Demo.ipynb` và ảnh chụp giao diện React của phiên chạy này.
- **Benchmark** ngày 24/10/2026, tại commit `ff49f59`: chạy ba kịch bản tải cao và bốn kịch bản phục hồi. Số liệu được lấy từ thư mục kết quả `bench-20261024T182433Z` do bộ công cụ benchmark tạo ra.

Các thay đổi giữa hai lần chạy gồm bổ sung cache `persist` cho kết quả của builder, ghi log thời gian theo từng giai đoạn và bổ sung phần benchmark. Các thay đổi này giữ nguyên ngữ nghĩa xử lý dữ liệu.

Bảng 16: Môi trường chạy thực nghiệm (Google Colab, profile colab-native)

Hạng mục	Giá trị quan sát
Hệ điều hành / kiến trúc	Linux, x86_64, chạy với quyền root (Cassandra khởi động với cờ `-R`)
CPU	8 CPU logic (AMD EPYC 7B12)
RAM	54 750 396 416 B (≈ 51,0 GiB); khả dụng lúc kiểm tra ≈ 49,4 GiB
Đĩa trống	216 798 367 744 B (≈ 201,9 GiB)
Phần mềm	Python 3.11.17, Temurin 17.0.20.1+1, Kafka 4.2.2, Cassandra 5.0.9, PySpark 3.5.7, Node.js 22.21.0, cloudflared 2026.10.0
Cấu hình	Spark `local[2]`, driver 16 GiB, whole-stage codegen tắt; Kafka heap 1 GiB; Cassandra heap 4 GiB + direct 2 GiB
Phạm vi live	10 tuyến xe buýt; `gtfs_version 8dc7ee14...6c6205`; `scope_id de5bf686d21385a5`; múi giờ `America/New_York`

4.3 Kết quả

4.3.1 Bootstrap và hạ tầng

Bảng 17 tổng hợp thời gian dựng môi trường. Trong bước bootstrap, hệ thống tải năm gói phần mềm và đối chiếu checksum với thông tin chính thức: JDK dùng SHA-256 từ Adoptium API, Kafka dùng SHA-512, Cassandra và Node.js dùng SHA-256, còn cloudflared dùng SHA-256 trong release notes. Cả năm gói đều có checksum trùng khớp. Tệp `uv.lock` được tạo trên Colab với `lock_origin=generated_on_colab`, sau khi công cụ xác định phiên bản cho 69 gói phụ thuộc. Công cụ Doctor ghi nhận PASS ở mọi kiểm tra và một WARN do `uv.lock` chưa được commit vào repository.

Bảng 17: Thời gian dựng môi trường trên Colab

Bước	Thời gian	Kết quả
Bootstrap (uv, Python, JDK, Kafka, Cassandra, Node, biên dịch React, connector)	104,8 s	PASS, chạy lần đầu, chưa có cache
Doctor / preflight	0,6 s	PASS (1 WARN: `lock_origin`)
Kafka KRaft sẵn sàng	3,7 s	READY, khởi tạo cluster mới
Cassandra sẵn sàng	42,6 s	READY, phiên bản 5.0.9
Khởi tạo hạ tầng (topic + CQL migration)	9,8 s	5 topic CREATED, migration `0001_initial` APPLIED, TTL probe PASS

4.3.2 Kiểm thử unit, runtime và Spark

Lệnh `pytest -m 'unit or runtime or spark'` hoàn tất với **940 passed, 1 warning** sau 995,68 s, tương đương 16 phút 35 giây. Cảnh báo duy nhất là `StarletteDeprecationWarning` từ `fastapi.testclient`, không liên quan đến logic của dự án. Các kiểm thử Spark chiếm phần lớn thời gian do mỗi test phải khởi tạo và giải phóng cache trên Spark session `local[1]`. Kết quả này kiểm chứng các phép biến đổi ở mức unit; việc phối hợp giữa các thành phần vẫn cần được đánh giá bằng kiểm thử tích hợp.

4.3.3 Golden smoke end-to-end

Golden smoke đưa bộ dữ liệu golden qua toàn bộ hệ thống: publisher → Kafka → Spark → Cassandra → API. Sau mỗi bước, kết quả đọc từ API được đối chiếu với `expected.json`. Cả năm bước đều **PASS**, không có kiểm tra nào thất bại. Thời gian và kết quả từng bước được trình bày trong Bảng 18.

Bảng 18: Kết quả golden smoke (run `5cece312...`, tổng 266,2 s)

Giai đoạn	Thời gian	Kết quả
Sinh fixture (`synth-generate golden`)	0,4 s	exit 0
Nạp dữ liệu tĩnh (Spark batch + Cassandra)	42,5 s	exit 0, phạm vi dữ liệu tĩnh đã được công bố
Bộ xử lý / API / bộ phát sẵn sàng	10,0 / 2,2 / 1,0 s	READY
Bước S1 (ba feed đầu tiên)	165 s	PASS
Bước S2 (VP mới)	25 s	PASS
Bước S3 (phát lại dữ liệu trùng)	7 s	PASS
Bước S4 (dữ liệu trùng, đến muộn)	4 s	PASS
Bước S5 (chỉ tiến đồng hồ)	5 s	PASS

Thời gian trong bảng được tính từ lúc publisher đến điểm đồng bộ cho tới khi toàn bộ kiểm tra của bước đó PASS. Vì vậy, số liệu này phản ánh thời gian hoàn tất bước kiểm thử, khác với độ trễ từ lúc phát dữ liệu đến lúc dữ liệu có thể được đọc qua API (visibility latency). Bước S1 mất 165 s do bao gồm các micro-batch đầu tiên sau khi JVM khởi động. Spark phải lập kế hoạch và thực thi lần đầu cho cả ba feed, trong đó nhánh dự báo có kế hoạch xử lý phức tạp nhất. Các bước còn lại mất 4–25 s; S3 và S4 nhanh nhất vì dữ liệu trùng hoặc đến muộn không tạo generation mới. Qua toàn bộ pipeline, kết quả xác nhận các yêu cầu chính của FR05, FR07 và FR10: số xe không cộng dồn giữa các snapshot, mỗi chuyến chỉ có một target để tính KPI, bản ghi trùng không làm tăng lịch sử, con trỏ không lùi khi snapshot đến muộn và độ mới được cập nhật theo đồng hồ dù không có dữ liệu mới.

4.3.4 Thu thập trực tiếp dữ liệu MBTA

Sau golden smoke, notebook tiếp tục chạy một phiên thu thập trực tiếp trong 600 giây trên 10 tuyến xe buýt. Bảng 19 tóm tắt các giai đoạn của phiên chạy.

Bảng 19: Các giai đoạn phiên thu thập trực tiếp (run `1908043e...`)

Giai đoạn	Thời gian	Ghi chú
Tải GTFS ZIP của MBTA	≈ 1 s	`static_download ok`
Spark trích xuất và kiểm tra	≈ 87 s	Kiểm tra dữ liệu tĩnh PASSED; 10/10 tuyến hợp lệ, không có tuyến bị loại
Ghi các bảng chiều dạng Parquet	≈ 13 s	`static_write_parquet ok`
Ghi danh mục dữ liệu tĩnh vào Cassandra	≈ 260 s	riêng `stop_times_by_trip` ≈ 253 s
Tổng thời gian nạp dữ liệu tĩnh	**367,6 s**	
Khởi động bộ xử lý / bộ thu thập	11,0 / 1,0 s	Dùng lại tiến trình API của golden smoke
Chờ generation READY đầu tiên	38,6 s	VP READY; `/health/ready`, `/runs`, `/quality` trả 200

Việc nạp GTFS thực tế của MBTA mất thời gian gần gấp 9 lần so với bộ fixture. Khoảng 70% thời gian nạp

dành cho việc ghi bảng `stop_times_by_trip` vào Cassandra, cho thấy đây là bước chậm nhất của quá trình nạp dữ liệu tĩnh trong lần chạy này. Hướng cải thiện được trình bày ở Phần 5.

4.3.5 Truy vết dữ liệu và độ trễ công bố

Lệnh `demo.py lineage` được chạy lúc 13:47:23Z để truy vết generation VP hiện hành. Kết quả **PASS**, với thông tin nhất quán giữa các tầng từ dữ liệu gốc đến API như trình bày trong Bảng 20.

Bảng 20: Truy vết snapshot `sn_33e812db...` (feed vehicle_positions)

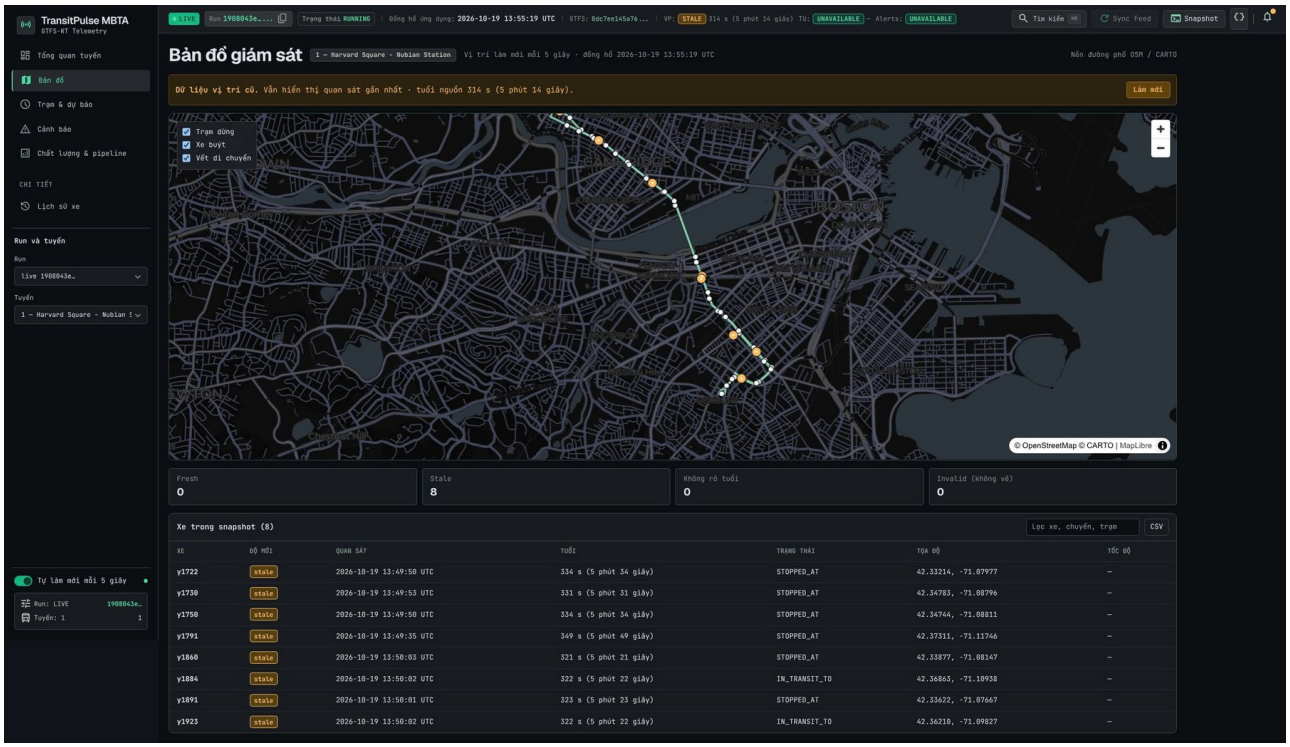
Tầng	Quan sát
Con trỏ	`is_current = true`; `generation_id gn_1689bd0c...`; `view_published_at` 13:47:11.842Z; `expires_at` 14:46:46.679Z
Manifest	`READY`, `publish_outcome = PUBLISHED`; `published_at` 13:46:37.017Z; `ready_at` 13:47:11.829Z; `visibility_latency_ms` = 34 825
Raw	Nội dung JSON 310 371 B, nén gzip còn 33 767 B; SHA-256 kỳ vọng và thực tế trùng khớp (`465ca68e...`)
Kafka	Topic `mbta.vehicle_positions.snapshots.v1`, partition 0, offset 5; envelope 40 512 B; `snapshot_id` và `capture_id` trong header khớp envelope
Batch	`first_batch_id` = 2
API	HTTP 200 trên `/vehicles`, trả đúng `snapshot_id` của con trỏ

Kết quả truy vết cho thấy các điểm sau:

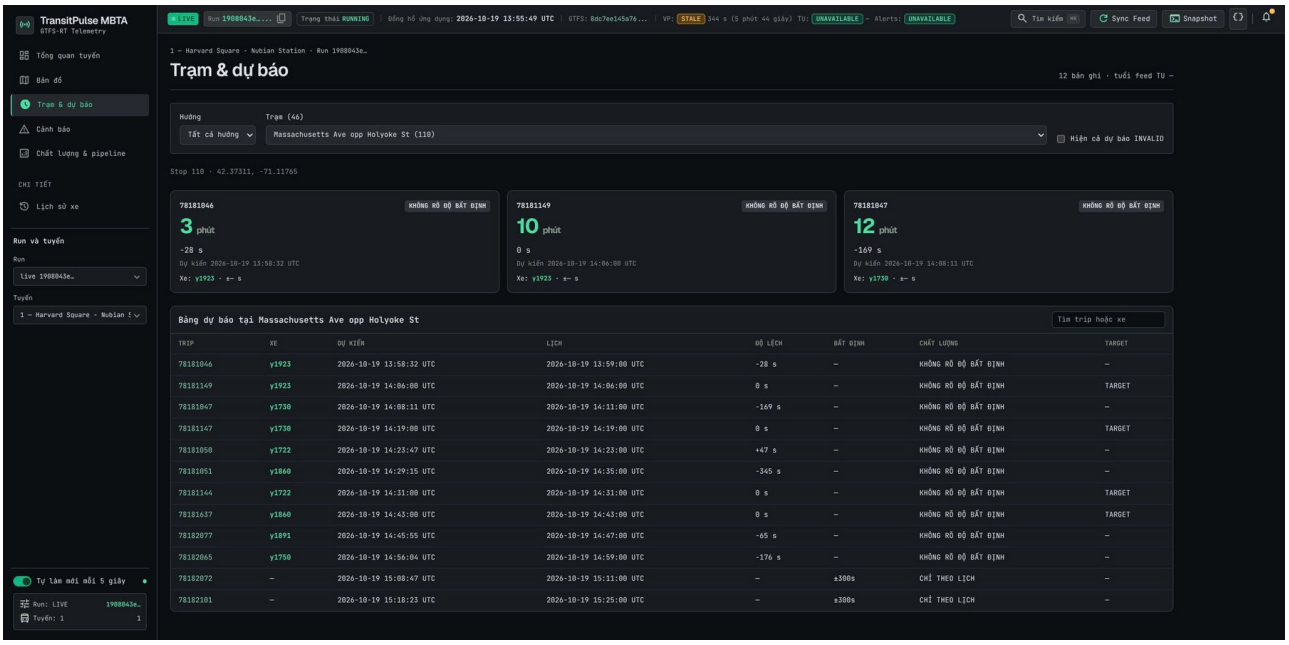
- **Lưu trữ và lọc theo phạm vi.** Nội dung Vehicle Positions của toàn mạng MBTA có kích thước 310 371 B. Sau khi nén gzip, kích thước giảm khoảng 9,2 lần. Khi chỉ giữ dữ liệu của 10 tuyến, thông điệp envelope trên Kafka còn 40 512 B, tương đương khoảng 13% nội dung gốc.
- **Thời gian ở từng bước xử lý.** Visibility latency của mẫu này là 34,8 s. Theo các mốc trong manifest, từ lúc generation đạt READY đến khi con trỏ được cập nhật bằng CAS chỉ mất 13 ms. Như vậy, gần như toàn bộ độ trễ phát sinh trước READY. Ở chế độ live, `metrics_as_of` được cố định trong manifest theo giờ máy lúc bộ xử lý bắt đầu xử lý snapshot, ở đây là 13:46:46.679Z. Dựa vào mốc này, có thể ước lượng khoảng 9,7 s từ lúc phát dữ liệu đến lúc bộ xử lý bắt đầu xử lý, bao gồm thời gian chờ trigger, chờ micro-batch trước và phân tích dữ liệu. Khoảng 25,2 s còn lại dành cho xây dựng generation, ghi Cassandra và công bố kết quả. Với generation VP đầu tiên của phiên chạy, khoảng thời gian từ lúc bắt đầu xây dựng đến lúc có thể đọc được là 34,7 s.
- **Đối chiếu với mục tiêu.** Độ trễ của mẫu đơn lẻ này **lớn hơn ngưỡng của mục tiêu p95 < 10 s**. Tại thời điểm truy vết, TU và Alerts chưa có generation READY, đồng thời dữ liệu curated của batch 2 chưa được commit. Điều này cho thấy micro-batch vẫn xử lý các feed còn lại sau khi VP đã được công bố. Mục 4.3.6 trình bày độ trễ lớn hơn của TU và Alerts trong cùng micro-batch; nguyên nhân được phân tích ở Mục 4.4.
- **Độ mới của nguồn.** Tại thời điểm truy vết, feed VP có tuổi 49,2 s, vẫn dưới ngưỡng stale 120 s.

4.3.6 Giao diện trong phiên thu thập trực tiếp

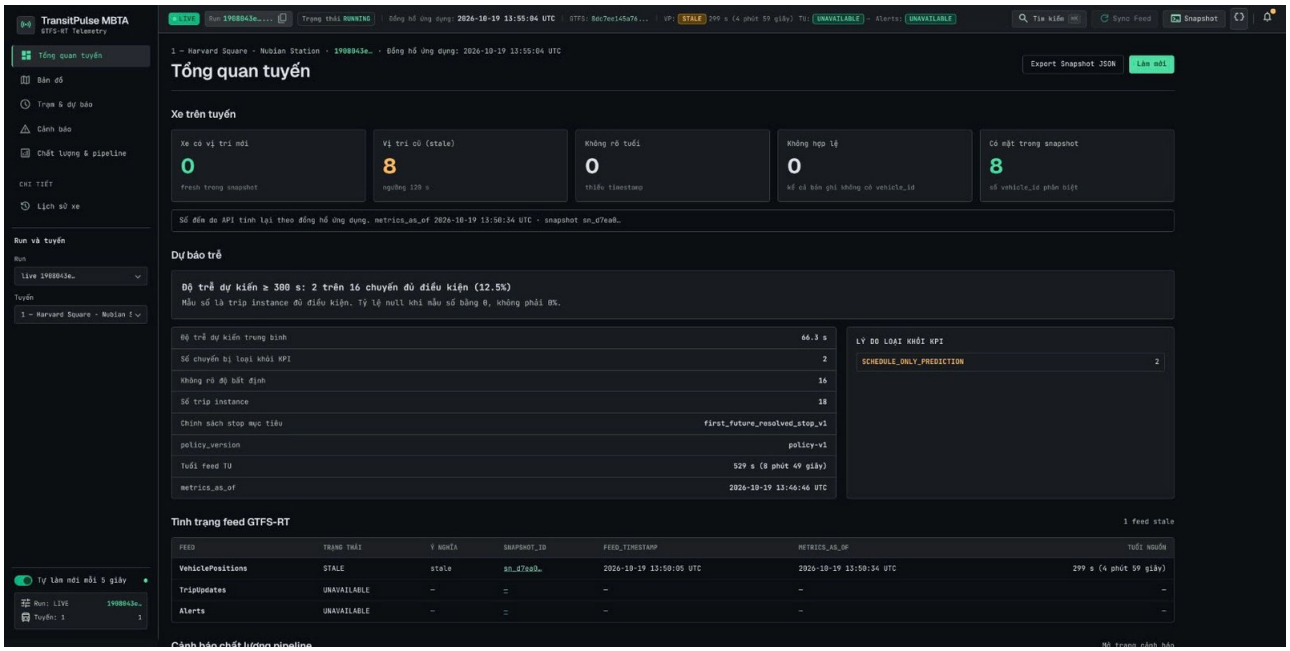
Hình 3, 4 và 5 ghi lại giao diện React trong phiên thu thập trực tiếp, với run `1908043e...` và tuyến 1 Harvard Square – Nubian Station. Các ảnh được chụp trong khoảng 13:55–13:56 UTC, gần cuối phiên chạy 600 s.



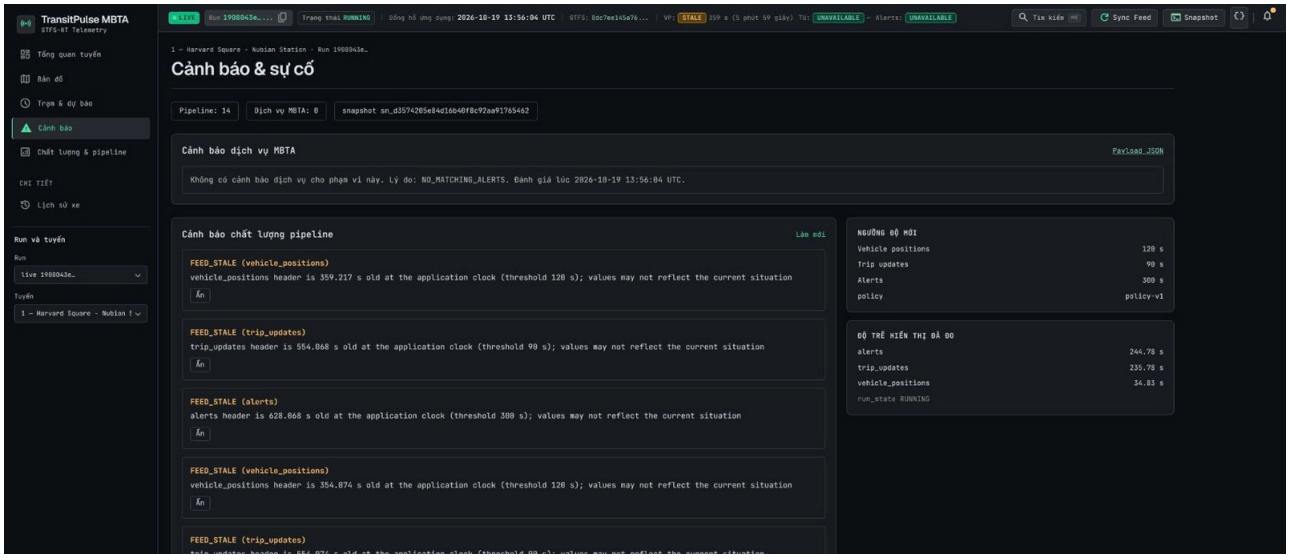
Hình 3: *Màn hình Bản đồ hiển thị 8 xe tuyến 1 trên nền CARTO/OpenStreetMap. Cả 8 xe đều ở trạng thái stale, với tuổi dữ liệu 321–349 s so với ngưỡng 120 s. Giao diện giữ vị trí xe và hiển thị thông báo "Dữ liệu vị trí cũ".*



Hình 4: *Màn hình Trạm & dự báo tại trạm 110 (Massachusetts Ave opp Holyoke St) hiển thị giờ dự kiến, giờ theo lịch, độ lệch có thể âm, nhãn chất lượng và cờ TARGET của từng chuyến. Hai dòng cuối là dự báo chỉ dựa trên lịch, với uncertainty bằng 300.*



(a) Màn hình Tổng quan tuyến hiển thị số xe theo mức độ mới, KPI độ trễ dự kiến kèm mẫu số và lý do loại trừ, cùng trạng thái của từng feed.



(b) Màn hình Cảnh báo trình bày riêng cảnh báo dịch vụ MBTA và cảnh báo chất lượng pipeline. Khung bên phải hiển thị visibility latency gần nhất của từng feed.

Hình 5: *Màn hình Tổng quan và Cảnh báo trên giao diện React; ảnh đã cắt phần dưới.*

Từ các ảnh chụp, có thể ghi nhận thêm bốn điểm trên dữ liệu thực tế:

- **Số lượng chuyến dùng để tính KPI khớp.** Tuyến 1 có 18 trip instance, trong đó 16 chuyến đủ điều kiện và 2 chuyến bị loại với lý do `SCHEDULE_ONLY_PREDICTION`. Các số đếm thỏa mãn trip_instances = eligible + $\sum$excluded. Có 2/16 chuyến có độ trễ dự kiến từ 300 s trở lên, tương ứng 12,5%; độ trễ dự kiến trung bình là 66,3 s. Cả 16 chuyến đủ điều kiện đều thiếu trường uncertainty, phù hợp với nhận định rằng dữ liệu xe buýt MBTA thường không có trường này.

- **Độ trễ của từng feed.** Endpoint `/pipeline/status` ghi nhận visibility latency gần nhất là 34,83 s cho VP, 235,78 s cho TU và 244,78 s cho Alerts. Giá trị VP khớp với số liệu lineage 34 825 ms. Hai generation VP và TU có cùng `metrics_as_of` 13:46:46 và cùng thuộc micro-batch 2. Thứ tự VP < TU < Alerts, cùng chênh lệch khoảng 200 s giữa VP và TU, cho thấy nhánh TU chiếm phần lớn thời gian xử lý của micro-batch.
- **Đánh giá độ mới khi pipeline xử lý chậm.** Lúc 13:55, generation VP hiện hành có `metrics_as_of` 13:50:34, dù bộ thu thập vẫn lấy dữ liệu mỗi 30 s. Trong khoảng 5 phút, bộ xử lý chưa công bố thêm generation VP nào. Khi nhận request, API tính lại tuổi dữ liệu và giao diện chuyển cả 8 xe sang stale, kèm cảnh báo `FEED_STALE`. Cơ chế đánh giá độ mới hoạt động đúng thiết kế. Tuy nhiên, việc dữ liệu trở nên cũ cũng cho thấy bộ xử lý không theo kịp tốc độ thu thập, phù hợp với kết quả benchmark ở Mục 4.4.
- **Lỗi hiển thị còn tồn tại.** Danh sách cảnh báo chất lượng pipeline lặp lại nhiều mục `FEED_STALE`, tổng cộng 14 mục cho 3 feed. Ngoài ra, thanh trạng thái phía trên báo TU và Alerts là `UNAVAILABLE`, trong khi các màn hình vẫn hiển thị KPI từ TU và snapshot cảnh báo hiện hành. Hàm `feedView` trong `frontend/src/format.js` gán trạng thái mặc định `unavailable` nếu response đang dùng không chứa feed tương ứng. Do đó, nguyên nhân nhiều khả năng nằm ở cách hiển thị trạng thái.

4.4 Benchmark

4.4.1 Phương pháp

Bộ công cụ tại `benchmarks/` đo thông lượng xử lý (throughput), độ trễ (latency), lượng dữ liệu chờ xử lý (backlog), mức sử dụng tài nguyên và thời gian phục hồi trên hệ thống native thực tế. Mỗi lần lặp tạo một run mới, khởi động bộ xử lý và chạy giai đoạn làm nóng (warm-up). Sau đó, công cụ đo trong một khoảng thời gian cố định, ngừng phát dữ liệu và chờ xử lý phần còn lại (drain) cho tới khi backlog bằng 0 hoặc hết thời gian chờ. Các kịch bản được liệt kê trong Bảng 21.

Bảng 21: Kịch bản benchmark

Kịch bản	Đầu vào	Nội dung đo
`replay_1x|10x|50x`	Bản ghi dữ liệu thu thập trực tiếp	Thông lượng, độ trễ, backlog, tài nguyên
`stress_100|500|1000`	Dữ liệu tải cao được sinh, 5 tuyến, 80/20 TU/VP	Như trên, ở 100/500/1000 bản ghi chuẩn hóa/s
`recovery_partial_write`	Golden + arm file	Mã thoát 86, khởi động lại cùng checkpoint, thời gian về VP2
`recovery_after_cas`	Golden + arm file	Như trên, gây lỗi sau CAS và trước commit
`recovery_cassandra_down`	Golden, dừng Cassandra	Batch thất bại, khởi động lại, thời gian trở về generation đúng
`recovery_collector_outbox`	Nguồn giả lập, không kết nối được broker, SIGKILL	Thời gian gửi tiếp từ outbox, số bản ghi mất/trùng

Đơn vị đo là *bản ghi chuẩn hóa* (normalized candidate record): một entity Vehicle Positions, một sự kiện arrival hoặc departure của Trip Updates, hoặc một selector của cảnh báo. Các chỉ số được tính từ dữ liệu thu được

trong lần chạy như sau:

- Các phân vị p50/p95/p99 của visibility latency được lấy từ `snapshot_manifest.visibility_latency_ms` cho các generation được đưa vào hàng đợi trong cửa sổ đo. Generation đến muộn, hết hạn hoặc chưa từng được công bố được đếm riêng.
- Thời gian xử lý batch được lấy từ `query_progress.jsonl`. Một batch được tính nếu thời gian chạy của nó có phần giao với cửa sổ đo.
- Độ trễ ghi Cassandra được lấy từ log của giai đoạn `cassandra_write`, với một mẫu cho mỗi generation.
- Độ trễ API được đo bằng 100 lần gọi `/api/v1/vehicles` sau giai đoạn drain, tính theo thời gian xử lý phía server.
- CPU và RSS được lấy mẫu bằng psutil mỗi 2 s. Backlog Kafka được lấy mẫu mỗi 5 s, bằng high watermark trừ end offset đã xử lý.

Các phân vị được tính bằng `statistics.quantiles(method="inclusive")`. Nếu số mẫu $n < 20$, p95 được ghi `null`; nếu $n < 100$, p99 được ghi `null`. Báo cáo luôn kèm số mẫu, giá trị nhỏ nhất và giá trị lớn nhất.

Trong lần chạy `bench-20261024T182433Z`, mỗi kịch bản được lặp một lần, với 180 s làm nóng, 180 s đo và tối đa 60 s chờ xử lý backlog. Các kịch bản tải cao bật whole-stage codegen cho bộ xử lý bằng `TP_SPARK_WHOLE_STAGE=true`. Cả bảy kịch bản đã chạy đều có trạng thái `MEASURED`. Kịch bản replay chưa được chạy do chưa có bản ghi dữ liệu MBTA thực tế.

4.4.2 Kết quả stress

Ở cả ba mức tải, publisher giữ tốc độ 0,4 snapshot/s, tương ứng một snapshot VP và một snapshot TU mỗi 5 s. Số bản ghi trong mỗi snapshot được tăng theo mức tải. Kết quả được tổng hợp trong Bảng 22 và 23.

Bảng 22: Thông lượng và backlog khi tăng tải bằng dữ liệu sinh, không phải lưu lượng MBTA

Chỉ số	stress_100	stress_500	stress_1000
Bản ghi đầu vào khai báo (bản ghi/s)	100	500	1 000
Snapshot đầu vào (snapshot/s)	0,40	0,40	0,40
Lưu lượng Kafka (kB/s)	15,4	75,2	149,9
Bản ghi đã xử lý (bản ghi/s)	6,67	25,0	50,0
Tỉ lệ so với đầu vào	6,7%	5,0%	5,0%
Generation READY trong cửa sổ	12	9	3
Dòng Cassandra đã ghi	2 460	9 045	10 017
Micro-batch hoàn tất trong cửa sổ	0	0	0
Backlog tối đa VP / TU / control (message)	71 / 71 / 429	71 / 71 / 428	71 / 72 / 430
Xử lý hết backlog trong 60 s	Không	Không	Không

Bảng 23: Độ trễ và tài nguyên khi tăng tải

Chỉ số	stress_100	stress_500	stress_1000
Visibility latency (số mẫu)	– (0)	– (0)	– (0)
Ghi Cassandra mỗi generation, p50 (số mẫu)	0,74 s (12)	0,89 s (9)	1,80 s (3)
Ghi Cassandra, min – max	0,66 – 0,85 s	0,85 – 1,04 s	1,05 – 1,80 s
API /vehicles p50 / p95 / p99 (ms)	12,7 / 15,0 / 16,6	13,9 / 16,6 / 66,2	20,0 / 25,3 / 70,3
CPU bộ xử lý, trung bình / tối đa (% một lõi)	159 / 273	154 / 286	217 / 644
CPU Cassandra / Kafka trung bình (% một lõi)	6,7 / 2,7	5,5 / 2,4	4,7 / 2,3
RSS bộ xử lý tối đa (GiB)	10,1	17,9	18,0
Tổng RSS tối đa (GiB)	15,0	22,8	23,0

Khả năng theo kịp đầu vào. Ở cả ba mức tải, bộ xử lý chỉ xử lý được 5–6,7% số bản ghi đầu vào. Backlog của mỗi topic snapshot tăng đến khoảng 71 thông điệp, gần bằng toàn bộ số snapshot được phát trong cửa sổ đo, và vẫn chưa được xử lý hết sau 60 s drain. Bộ xử lý dành suốt 180 s đo để xử lý một micro-batch dài, không có micro-batch nào hoàn tất nên công cụ không thu được mẫu thời gian batch. Tương tự, không snapshot nào được đưa vào hàng đợi trong cửa sổ đo có thể đọc được trước khi hết giai đoạn drain, nên visibility latency không có mẫu. Với các snapshot này, độ trễ lớn hơn khoảng thời gian từ lúc vào hàng đợi đến cuối giai đoạn drain, tức từ 60 s đến 240 s, cao hơn nhiều so với mục tiêu 10 s. Các generation đạt READY trong cửa sổ đo đều thuộc những snapshot đã vào hàng đợi từ giai đoạn làm nóng.

Thời gian xây dựng generation trên Spark. Khi chia thời lượng cửa sổ đo cho số generation READY, thời gian tương ứng là khoảng 15 s, 20 s và 60 s mỗi generation. Trong đó, p50 thời gian ghi Cassandra chỉ từ 0,74–1,80 s, chiếm dưới 5%. Cassandra sử dụng trung bình dưới 7% một lõi CPU và Kafka dưới 3%, còn bộ xử lý dùng khoảng 1,5–2,2 lõi, gần hết hai luồng của Spark `local[2]`. Các số liệu này cho thấy nút thắt chủ yếu nằm ở bước xây dựng generation trên Spark cho từng snapshot.

Bộ sinh dữ liệu tải cao phân chia bản ghi theo tỉ lệ 80/20 giữa TU và VP. Với tốc độ mục tiêu R, mỗi snapshot VP chứa R bản ghi và mỗi snapshot TU chứa $4R$. Từ số bản ghi đã xử lý và số generation, có thể xác định duy nhất số generation của từng feed:

- `stress_100`: 12 generation đều là VP ($12 \times 100 = 1\,200$);
- `stress_500`: 9 generation đều là VP ($9 \times 500 = 4\,500$);
- `stress_1000`: 1 VP và 2 TU ($1\,000 + 2 \times 4\,000 = 9\,000$). Có 1 000 bản ghi duy nhất, đúng bằng số quan sát trong một snapshot VP.

Như vậy, ở hai mức tải thấp, bộ xử lý chưa hoàn tất generation TU nào trong 180 s đo. Dựa trên hai điểm đo của VP, có thể ước lượng sơ bộ thời gian cho mỗi generation gồm khoảng 14 s chi phí cố định và thêm khoảng 1,25 s cho mỗi 100 bản ghi. Phần chi phí cố định đến từ việc lập kế hoạch và chạy nhiều Spark job cho từng snapshot, lớn hơn đáng kể so với phần tăng theo lượng dữ liệu. Kết hợp với quan sát ở phiên live, trong đó TU chậm hơn VP khoảng 200 s trong cùng micro-batch, nhánh TU là phần tốn thời gian xử lý nhất của pipeline.

Thời gian phản hồi API. Khi bộ xử lý bão hòa, endpoint `/vehicles` vẫn có p50 trong khoảng 13–20 ms và p95 trong khoảng 15–25 ms. Mỗi request đọc con trỏ hiện hành rồi truy vấn một số phân vùng có giới hạn. Generation bất biến giúp tách việc đọc khỏi quá trình ghi. Ở hai mức tải cao, p99 tăng đến khoảng 66–70 ms. Do chỉ có 100 mẫu, p99 gần với giá trị lớn nhất và dễ thay đổi khi xuất hiện một vài request chậm.

Mức sử dụng bộ nhớ. RSS của bộ xử lý đạt tối đa 18 GiB, gồm heap driver 16 GiB, Python worker và bộ nhớ ngoài heap. Tổng RSS của hệ thống khoảng 23 GiB, nằm trong ngân sách bộ nhớ của profile `colab-native`.

4.4.3 Kết quả phục hồi

Trong mỗi kịch bản, bộ công cụ chủ động gây một lỗi trên tiến trình do nó quản lý khi đang chạy golden fixture. Thời gian phục hồi được tính từ lúc bắt đầu khởi động lại Cassandra hoặc bộ xử lý đến khi generation VP2 trở thành hiện hành và khớp `expected.json`. Riêng với outbox, thời gian được tính từ lúc khởi động lại bộ thu thập đến khi broker xác nhận toàn bộ bản ghi đang chờ gửi. Cột "Chờ" thể hiện khoảng thời gian từ lúc gây lỗi đến lúc công cụ phát hiện lỗi và bắt đầu phục hồi; khoảng này được tính riêng.

Bảng 24: Kết quả bốn kịch bản phục hồi (một lần lặp mỗi kịch bản)

Kịch bản	Lỗi được tạo	Mã thoát	Chờ	Phục hồi	Khớp	Mất / trùng
`recovery_partial_write`	Dừng sau khi ghi một phần	86	26,8 s	**43,5 s**	Có	0 / 0
`recovery_after_cas`	Dừng sau CAS, trước commit	86	25,3 s	**15,4 s**	Có	0 / 0
`recovery_cassandra_down`	Dừng Cassandra	1	11,0 s	**49,5 s**	Có	0 / 0
`recovery_collector_outbox`	SIGKILL bộ thu thập, không kết nối được broker	SIGKILL	2,0 s	**1,0 s**	Có	0 / 0

Cả bốn kịch bản đều khớp kết quả kỳ vọng, với số bản ghi mất và trùng đều bằng 0. Kết quả này kiểm chứng hoạt động của giao thức công bố generation ở Mục 3.8 và cơ chế outbox qua toàn bộ hệ thống:

- **Dừng đột ngột sau CAS**: thời gian phục hồi là 15,4 s, nhanh nhất trong các kịch bản phục hồi bộ xử lý. Khi Spark xử lý lại micro-batch, manifest đã ở trạng thái READY và con trỏ đã trỏ đúng snapshot. Vì vậy, lần xử lý lại được ghi nhận thành công mà không làm thay đổi kết quả.
- **Dừng đột ngột sau khi ghi một phần**: hệ thống mất 43,5 s để xây dựng và ghi lại generation theo các tham số đã cố định, sau đó mới đánh dấu READY. Trong thời gian này, API tiếp tục đọc generation cũ.
- **Cassandra ngừng hoạt động**: thời gian phục hồi 49,5 s bao gồm cả việc khởi động lại Cassandra. Để đối chiếu, Cassandra mất 42,6 s để khởi động trong lần dựng hạ tầng ở Bảng 17.
- **Outbox**: bộ thu thập bị SIGKILL khi vẫn còn bản ghi chờ gửi. Sau khi khởi động lại, bộ thu thập gửi hết các bản ghi này trong 1,0 s, không ghi nhận bản ghi mất hoặc trùng.

Do mỗi kịch bản mới được chạy một lần, các số liệu trên chỉ phản ánh từng lần quan sát và chưa đủ để mô tả phân phối thời gian phục hồi.

Bộ kiểm thử tích hợp IT01–IT16 dưới dạng pytest chưa được chạy trong lần Run All đã ghi nhận, do `RUN_INTEGRATION_TESTS=False`. Các kịch bản phục hồi ở trên kiểm tra cùng nhóm tính chất với IT10–IT12 và việc gửi tiếp từ outbox sau khi khởi động lại, đồng thời đối chiếu với cùng tệp `expected.json`.

4.5 Tổng hợp mức độ đáp ứng

Bảng 25 đối chiếu các yêu cầu với kết quả hiện có và nêu phần còn thiếu. Việc đánh giá phân biệt rõ những nội dung đã được kiểm chứng qua toàn bộ hệ thống, những nội dung mới có kết quả ở mức unit và những kịch bản chưa chạy.

Bảng 25: Mức độ đáp ứng yêu cầu theo bằng chứng hiện có

Yêu cầu	**Bằng chứng end-to-end**	**Còn thiếu**
FR01 GTFS tĩnh	Nạp dữ liệu tĩnh golden và live PASS; 10/10 tuyến hợp lệ	–
FR02–FR03 Thu thập, Kafka	Phiên live; truy vết raw → Kafka khớp checksum; outbox phục hồi sau SIGKILL, 0 mất, 0 trùng	IT05, IT13 chưa chạy
FR04–FR07 Xử lý, KPI	Golden S1–S5 PASS qua API; 940 kiểm thử PASS	IT06–IT08 chưa chạy
FR08 Cảnh báo	Golden S1–S5 (alerts) PASS	IT09 chưa chạy
FR10 Generation, con trỏ	Golden S3 (trùng), S4 (muộn) PASS; truy vết READY → CAS; phục hồi sau ghi một phần, sau CAS và sau khi dừng Cassandra đều khớp kỳ vọng	IT15 (TTL) chưa chạy
FR11 Replay	Golden dùng bộ phát lại theo đồng hồ nguồn	Chưa phát lại bản ghi dữ liệu MBTA thực tế
FR12, NFR05 API, truy vết	API 200, truy vết nhất quán, giao diện notebook 5 tab, giao diện React qua tunnel (Hình 3–5)	Sửa lỗi hiển thị trạng thái feed và cảnh báo lặp; chưa có test UI
NFR02 Phục hồi	4/4 kịch bản phục hồi khớp kỳ vọng, 0 mất, 0 trùng; 1,0–49,5 s	Mới một lần lặp; IT10–IT12 dạng pytest chưa chạy
NFR03 Độ trễ	**Chưa đạt.** Live: VP 34,8 s, TU 235,8 s, Alerts 244,8 s; stress: chỉ xử lý 5–6,7% đầu vào, độ trễ > 60 s	Tăng thông lượng bộ xử lý; benchmark replay chưa chạy
NFR08 Giới hạn truy vấn	API `/vehicles` p95 15–25 ms khi bộ xử lý bão hòa	–
NFR06 Tái lập	Checksum của 5 gói phần mềm khớp; đã tạo `uv.lock`	Commit tệp `uv.lock`

5 Kết luận

5.1 Tổng kết

Báo cáo đã trình bày quá trình xây dựng TransitPulse MBTA, từ thu thập và xử lý đến cung cấp dữ liệu xe buýt MBTA theo thời gian thực bằng Kafka, Spark Structured Streaming và Cassandra. Trọng tâm của đồ án là duy trì kết quả nhất quán khi dữ liệu nguồn có lỗi, sự kiện có thể được nhận nhiều lần và các thành phần không có giao dịch chung. Các kết quả chính gồm:

- **Cơ chế công bố generation**: dữ liệu của mỗi generation được giữ bất biến, còn các tham số trong manifest được cố định từ lần tạo đầu tiên. Hệ thống kiểm tra số dòng trước khi chuyển sang READY và cập nhật con trỏ bằng compare-and-set theo thứ tự dữ liệu nguồn. Cơ chế này giúp API tránh đọc dữ liệu ghi dở, giữ KPI không tăng do bản ghi trùng và ngăn snapshot cũ thay thế trạng thái mới. Trong lần chạy thực tế, khoảng thời gian từ READY đến khi con trỏ được cập nhật bằng CAS là 13 ms.
- **Mô hình lưu trữ theo nhu cầu truy vấn**: Cassandra sử dụng 17 bảng, không cần `ALLOW FILTERING` hay counter. Các bảng phục vụ truy vấn được phân vùng theo `snapshot_id` và tự xóa dữ liệu khi hết TTL.
- **Cách tính chỉ số rõ ràng**: độ mới của dữ liệu được tính lại khi API trả kết quả; giờ GTFS được chuyển đổi có xét ngày đổi giờ mùa hè; mỗi chuyến chỉ đóng góp một sự kiện mục tiêu vào KPI. Tử số và mẫu số được công bố kèm kết quả. Khi mẫu số bằng 0, kết quả KPI nhận giá trị `null` thay vì 0%.
- **Kết quả kiểm chứng trên Google Colab**: bước khởi tạo kiểm tra checksum của 5 thành phần tải về; 940 kiểm thử đơn vị, môi trường chạy và Spark đều đạt (PASS). Kiểm thử golden smoke từ đầu đến cuối đạt cả 5 bước, bao gồm dữ liệu trùng, dữ liệu đến muộn và tăng đồng hồ nghiệp vụ. Phiên chạy trực tiếp trên 10 tuyến MBTA cũng cho phép truy vết nhất quán từ API về checksum của dữ liệu gốc.
- **Khả năng phục hồi khi có lỗi**: cả bốn kịch bản đo phục hồi đều cho kết quả như kỳ vọng, không mất hoặc trùng bản ghi, với thời gian từ 1,0–49,5 s. Các kịch bản gồm dừng đột ngột sau khi ghi một phần dữ liệu, dừng sau CAS, Cassandra ngừng hoạt động và bộ thu thập bị dừng khi outbox còn bản ghi chờ gửi.
- **Thời gian phản hồi của API**: trong các phép đo, p95 duy trì ở mức 15–25 ms ngay cả khi bộ xử lý bão hòa. Kết quả này gắn với cách đọc dữ liệu theo phân vùng có giới hạn và sử dụng generation bất biến để tách quá trình đọc khỏi quá trình ghi.

5.2 Hạn chế

Hạn chế về hiệu năng. Hạn chế lớn nhất là **chưa đạt mục tiêu p95 visibility latency $<$ 10 s** trên Colab, tức thời gian chờ để dữ liệu có thể được truy vấn còn cao. Ở cả ba mức tải kiểm thử, bộ xử lý chỉ xử lý được 5–6,7% lượng bản ghi đầu vào và chưa giải quyết hết dữ liệu tồn đọng. Trong phiên chạy trực tiếp, TU và Alerts cần khoảng 4 phút để có thể được đọc qua API. Dữ liệu VP đã chuyển sang trạng thái cũ (stale) khi micro-batch tiếp theo còn chưa hoàn tất. Kết quả đo cho thấy nút thắt nằm ở chi phí cố định lớn khi Spark

tạo kết quả cho từng snapshot, đặc biệt ở nhánh TU; Kafka và Cassandra chưa phải các thành phần giới hạn hiệu năng trong các phép đo này.

Hạn chế về kiểm chứng.

- Các kiểm thử tích hợp IT01–IT16 bằng pytest và phép đo hiệu năng khi phát lại dữ liệu MBTA thực tế chưa được thực hiện. Mỗi kịch bản đo hiệu năng mới chạy một lần, và cấu hình Docker Compose cũng chưa được chạy thử.
- Phiên demo và golden smoke sử dụng commit `4d6669e`, trong khi phép đo hiệu năng sử dụng commit `ff49f59`. Tệp `uv.lock` được tạo trên Colab nhưng chưa đưa vào commit, nên phiên bản các thư viện phụ thuộc gián tiếp có thể khác nhau giữa hai lần khởi tạo môi trường.
- Ảnh chụp giao diện React cho thấy thanh trạng thái báo `UNAVAILABLE` chưa đúng đối với feed không có trong phản hồi đang dùng, đồng thời danh sách cảnh báo xử lý còn lặp nội dung. Khi rà soát mã nguồn, nhóm cũng nhận thấy màn hình trạm có thể giữ dữ liệu ETA cũ nếu người dùng tiếp tục xem cùng một trạm; trường hợp này chưa được tái hiện khi chạy.

Giới hạn trong phạm vi thiết kế.

- Hệ thống sử dụng một Kafka broker và một Cassandra node với hệ số sao chép RF1. Cấu hình này chưa đáp ứng tính sẵn sàng cao và không chịu được sự cố mất máy chủ.
- Hệ thống bảo đảm giao nhận at-least-once, kết hợp ghi và công bố dữ liệu theo cách idempotent. Chưa có bảo đảm exactly-once xuyên suốt Kafka, Parquet và Cassandra. Con trỏ điều khiển việc công bố riêng cho từng feed, nên ba feed chưa được cập nhật trong một giao dịch nguyên tử chung.
- Thời hạn lưu dữ liệu TTL được tính theo đồng hồ hệ thống và vẫn tiếp tục khi quá trình phát lại đang tạm dừng.
- ETA được lấy từ MBTA. Dự án chưa có mô hình ETA riêng và chưa đánh giá độ chính xác do thiếu dữ liệu đối chứng. Dữ liệu xe buýt MBTA thường thiếu trường uncertainty; trong phiên chạy trực tiếp trên tuyến 1, cả 16/16 chuyến đủ điều kiện tính KPI đều thiếu trường này.
- Môi trường Colab hoạt động theo phiên, nên máy ảo có thể bị thu hồi và dữ liệu trong thư mục làm việc có thể mất khi máy ảo được đặt lại. Liên kết Cloudflare quick tunnel được truy cập công khai và chưa có xác thực.

5.3 Hướng phát triển

1. **Ưu tiên tăng năng lực xử lý**: gộp các snapshot cùng feed trong một micro-batch để xử lý bằng một lần chạy Spark, qua đó chia sẻ chi phí cố định khoảng 14 s mỗi generation cho nhiều snapshot. Các hướng khác gồm giảm số Spark action trên mỗi snapshot, tách VP và TU thành các truy vấn độc lập để nhánh

VP nhẹ hơn không phải chờ nhánh TU, chuẩn bị trước kế hoạch thực thi Spark và đo lại trên máy có nhiều executor.

2. **Bổ sung kết quả kiểm chứng**: đo lại khả năng chịu tải sau khi tối ưu, đo hiệu năng khi phát lại bản ghi MBTA thu trực tiếp trong 2 giờ và chạy IT01–IT16. Mỗi kịch bản cần được lặp nhiều lần, đồng thời đưa `uv.lock` vào commit để cố định phiên bản thư viện.

3. **Rút ngắn thời gian nạp dữ liệu tĩnh**: ghi `stop_times_by_trip` song song, bất đồng bộ với số tác vụ đồng thời có giới hạn, hoặc sử dụng Spark Cassandra Connector. Chỉ nạp các chuyến thuộc phạm vi theo dõi.

4. **Mở rộng hạ tầng**: thử nghiệm Kafka nhiều broker với nhiều partition theo feed, kết hợp Cassandra ba node, RF3 và `LOCAL_QUORUM`. Khả năng đáp ứng tải cần được đo lại trên cấu hình mở rộng.

5. **Đánh giá độ chính xác ETA**: đối chiếu dự báo với thời điểm xe thực tế đến trạm, ước lượng từ chuỗi Vehicle Positions, để xây dựng dữ liệu đối chứng gần đúng.

6. **Cải thiện an toàn và giao diện**: nâng Spark lên 3.5.8, bảo vệ tunnel bằng Cloudflare Access, sửa trạng thái feed và gộp các cảnh báo trùng trên giao diện React. Bổ sung kiểm thử hành vi cho việc làm mới ETA và hiển thị uncertainty.

6 Tài liệu tham khảo

Tham khảo

[1] Massachusetts Bay Transportation Authority, "MBTA GTFS and GTFS Realtime Documentation."
https://github.com/mbta/gtfs-documentation

[2] MobilityData, "GTFS Schedule Reference."
https://gtfs.org/documentation/schedule/reference/

[3] MobilityData, "GTFS Realtime Reference."
https://gtfs.org/documentation/realtime/reference/

[4] MobilityData, "GTFS Realtime Best Practices."
https://gtfs.org/documentation/realtime/realtime-best-practices/

[5] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia, "Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 2018.

[6] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache Spark: A Unified Engine for Big Data Processing," *Communications of the ACM*, vol. 59, no. 11, 2016.

[7] J. Kreps, N. Narkhede, and J. Rao, "Kafka: a Distributed Messaging System for Log Processing," in *Proceedings of the NetDB Workshop*, 2011.

[8] A. Lakshman and P. Malik, "Cassandra: A Decentralized Structured Storage System," *ACM SIGOPS Operating Systems Review*, vol. 44, no. 2, 2010.

[9] L. Lamport, "Paxos Made Simple," *ACM SIGACT News*, vol. 32, no. 4, 2001.

[10] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, "The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing," *Proceedings of the VLDB Endowment*, vol. 8, no. 12, 2015.

[11] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[12] C. Richardson, *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.

[13] Mã nguồn TransitPulse MBTA.
https://github.com/nqMinh-GR/de_btl